



CYBER SECURITY TECHNOLOGY ENGINEERING DEPARTMENT

MICROPROCESSOR SECOND STAGE

Lecture.1,2



Asst. Lect. Haider Saad

2024 - 2025

Introduction to 8086 Microprocessor

Microprocessors generally is categorized in terms of the maximum number of binary bits they can process i.e. their word length. There are so many manufacturers of Microprocessors, but only two companies have been produces popular microprocessors: *Intel* and *Motorola*. Table 1 lists some of types that belong to these companies (families) of microprocessors.

Type	Data bus width	Memory size
Intel family:		
8085	8	64K
8086	16	1M
80286	16	16M
80386EX , 80386DX	16 , 32	64M , 4G
80486DX4	32	4G + 16K cache
Pentium	64	4G + 16K cache
PentiumIII , Pentium4	64	64G+32K L1 cache +256 L2 cache
Motorola family:		
6800	8	64K
68060	64	4G + 16K cache

It is important to note that 80286, 80386, 80486, and Pentium-Pentium4 microprocessors are **upward compatible** with the 8086 Architecture. This mean that 8086/8088 code will run on the 80286, 80386, 80486, and Pentium Processors, but the reverse in not true if any of the new instructions are in use.

8086 Microprocessor

- 8086 is a 16 bit μ p, hence it provides 16-bit registers. It has a 20 bit address bus can access up to 2^{20} memory locations (1 MB) .
- It can support up to 64K I/O ports.
- It has multiplexed address and data bus AD0- AD15 and A16 – A19.

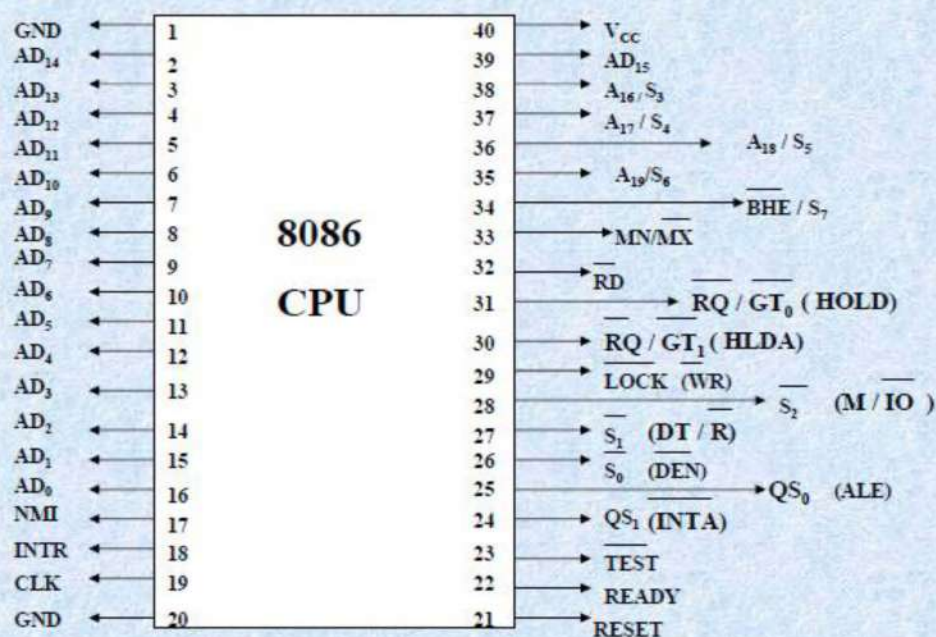
. 8086 is designed to operate in two modes, Minimum and Maximum. The minimum mode is selected by applying logic 1 to the MN / MX input pin. This is a single microprocessor configuration. The maximum mode is selected by applying logic 0 to the MN / MX input pin. This is a multi-microprocessors configuration.

- It can prefetching up to 6 instruction bytes from memory and queues them in order to speed up instruction execution.
- It requires +5V power supply.

Table 1-1: Evolution of Intel's Microprocessors

Product	8080	8085	8086	8088	80286	80386	80486
Year introduced	1974	1976	1978	1979	1982	1985	1989
Clock rate (MHz)	2 - 3	3 - 8	5 - 10	5 - 8	6 - 16	16 - 33	25 - 50
No. transistors	4500	6500	29,000	29,000	130,000	275,000	1.2 million
Physical memory	64K	64K	1M	1M	16M	4G	4G
Internal data bus	8	8	16	16	16	32	32
External data bus	8	8	16	8	16	32	32
Address bus	16	16	20	20	24	32	32
Data type (bits)	8	8	8, 16	8, 16	8, 16	8, 16, 32	8, 16, 32

Pin Diagram of 8086



Internal Architecture of 8086

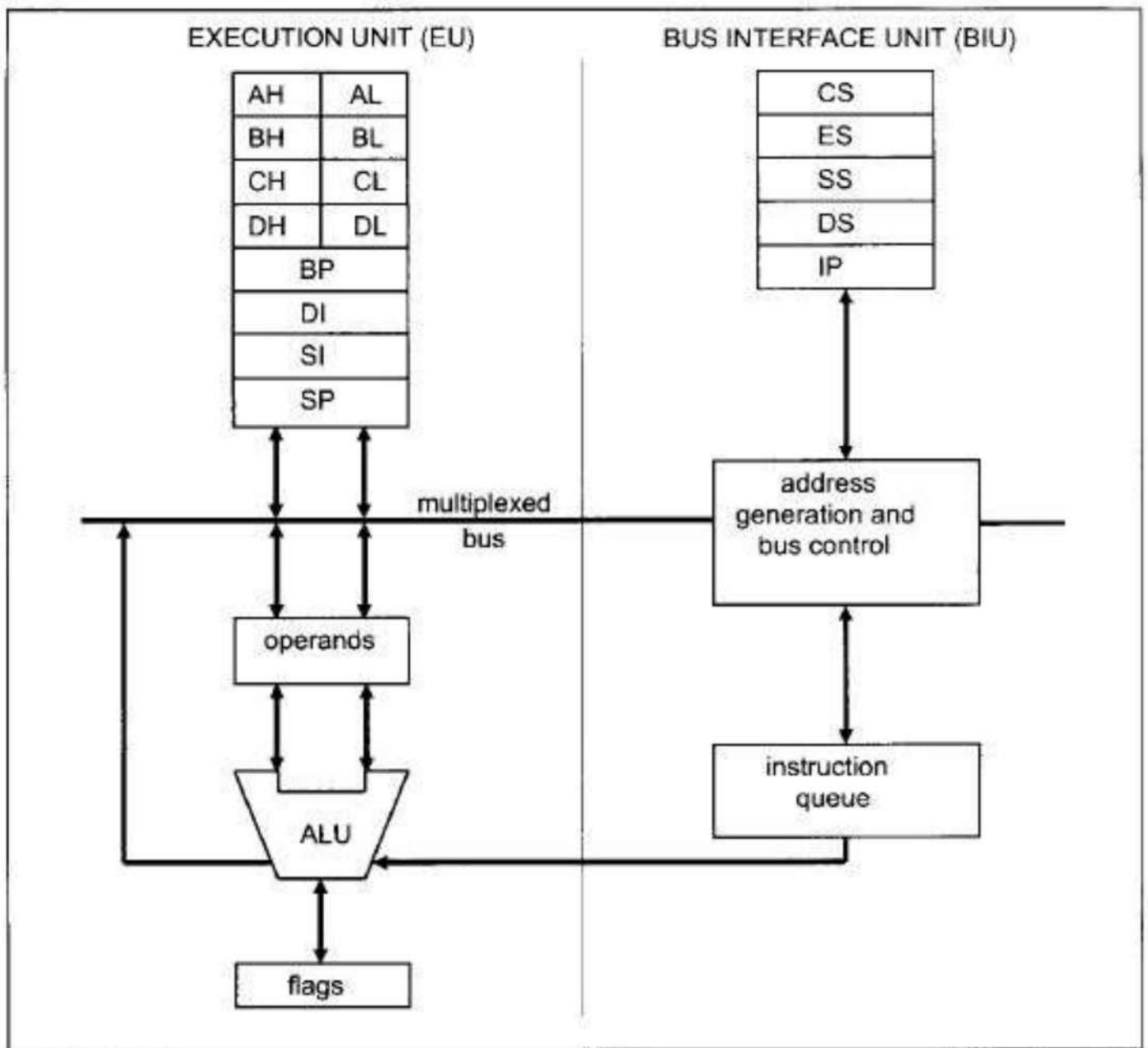
8086 from the software point of view has two blocks Bus Interface Unit (BIU) and Execution Unit (EU). Both units operate asynchronously to give the 8086 an overlapping instruction fetch and execution mechanism which is called as Pipelining. This results in efficient use of the system bus and system performance.

Bus Interfacr Unit (BIU)

- The BIU contains Instruction queue, Segment registers, Instruction pointer, address generator.
- It provides a full 16 bit bidirectional data bus and 20 bit address bus.
- It performs all bus operations such as instruction fetching, reading and writing operands for memory and calculating the addresses of the memory operands.
- The instruction bytes are transferred to the instruction queue. The BIU uses a mechanism known as an instruction stream queue to implement a *pipeline architecture*.

Piplining is implemented in 8086 μ p by splitting the internal structure into two sections BIU and EU. The two sections work simultaneously,

the BIU accessing memory to fetch and store data employing the queue buffer. At the same time, EU executes the instructions previously fetched.



Internal Block Diagram of the 8088/86 CPU

Pipelining

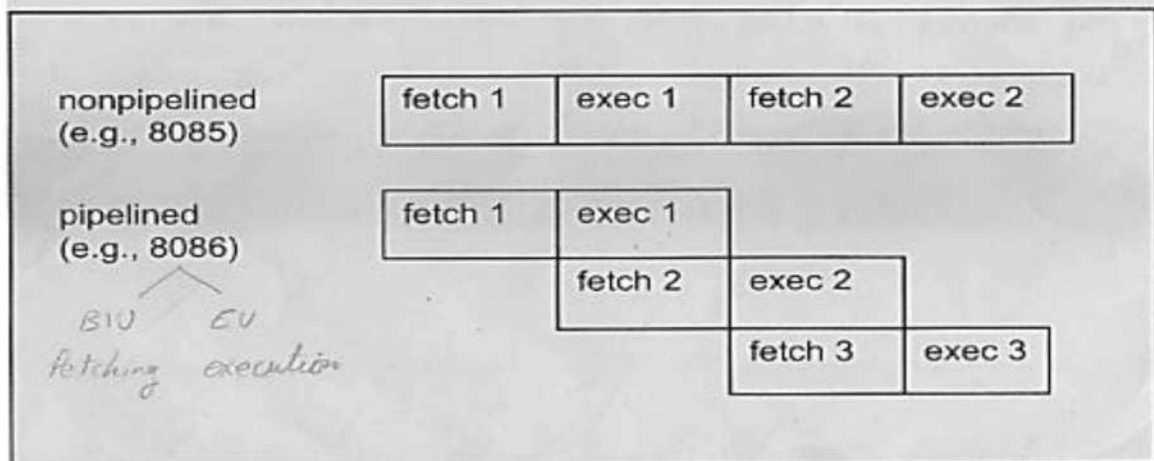


Figure 1-2. Pipelined vs. Nonpipelined Execution

The queue permits prefetching of up to six bytes of instruction code. Whenever the queue of the BIU is not full, it has room for at least two more bytes and at the same time the EU is not requesting it to read or write operands from memory, the BIU is free to look ahead in the program by prefetching the next sequential instruction.

- These prefetching instructions are held in its FIFO queue. With its 16 bit data bus, the BIU fetches two bytes in a single memory cycle. After a byte is loaded at the input end of the queue, it automatically shifts up through the FIFO to the empty location nearest the output.
- The BIU is also responsible for generating bus control signals such as those for memory read or write and I/O read or write.
- The BIU contains the following registers:

IP - the Instruction Pointer

CS - the Code Segment Register

DS - the Data Segment Register

SS - the Stack Segment Register

ES - the Extra Segment Register

Even though the 8086 has a 1Mbyte address space, not all this memory is active at one time. Actually, the 1Mbytes of memory are partitioned into 64Kbyte (65,536) *segments*. Each segment is assigned a *Base Address* that identifies its starting point (identify its lowest address byte-storage location). Only four of these 64Kbyte segments are active a time: the *code segment*, *stack segment*, *data segment*, and *extra segment*. The addresses of these four segments are held in the four segment registers.

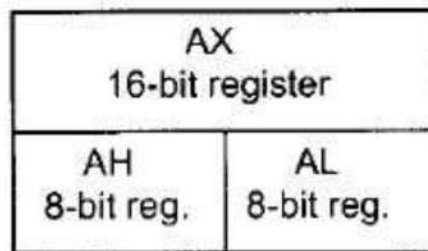
- The BIU also contains a dedicated adder which is used to generate the 20 bit physical address that is output on the address bus. This address is formed by adding an appended 16 bit segment address and a 16 bit offset address.
- For example, the physical address of the next instruction to be fetched is formed by combining the current contents of the code segment CS register and the current contents of the instruction pointer IP register.

EXECUTION UNIT (EU)

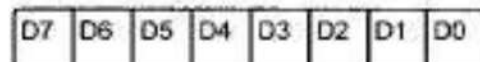
- The EU is responsible for decoding and executing all instructions.
- The EU extracts instructions from the top of the queue in the BIU, decodes them, generates operands if necessary, passes them to the BIU and requests it to perform the read or write bus cycles to memory or I/O and perform the operation specified by the instruction on the operands.
- During the execution of the instruction, the EU tests the status and control flags and updates them based on the results of executing the instruction.
- If the queue is empty, the EU waits for the next instruction byte to be fetched and shifted to top of the queue.
- When the EU executes a branch or jump instruction, it transfers control to a location corresponding to another set of sequential instructions. Whenever this happens, the BIU automatically resets the queue and then begins to fetch instructions from this new location to refill the queue.
- U contains Control circuitry, Instruction decoder, ALU, Pointer, Index and general registers, Flag register.

Registers

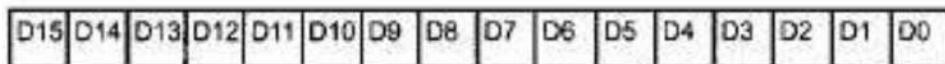
In the CPU, registers are used to store information temporarily. That information could be one or two bytes of data to be processed or the address of data. The registers of the 8088/86 fall into the six categories outlined in Table 1-2. The general-purpose registers in 8088/86 microprocessors can be accessed as either 16-bit or 8-bit registers. All other registers can be accessed only as the full 16 bits. In the 8088/86, data types are either 8 or 16 bits. To access 12-bit data, for example, a 16-bit register must be used with the highest 4 bits set to 0. The bits of a register are numbered in descending order, as shown below.



8-bit register:



16-bit register:



Different registers in the 8088/86 are used for different functions, and since some instructions use only specific registers to perform their tasks, the use of registers will be described in the context of instructions and their application in a given program. The first letter of each general register indicates its use. AX is used for the accumulator, BX as a base addressing register, CX is used as a counter in loop operations, and DX is used to point to data in I/O operations.

Table 1-2: Registers of the 8086/286 by Category

Category	Bits	Register Names
General	16	AX, BX, CX, DX
	8	AH, AL, BH, BL, CH, CL, DH, DL
Pointer	16	SP (stack pointer), BP (base pointer)
Index	16	SI (source index), DI (destination index)
Segment	16	CS (code segment), DS (data segment), SS (stack segment), ES (extra segment)
Instruction	16	IP (instruction pointer)
Flag	16	FR (flag register)

Note:

The general registers can be accessed as the full 16 bits (such as AX), or as the high byte only (AH) or low byte only (AL).

General data registers

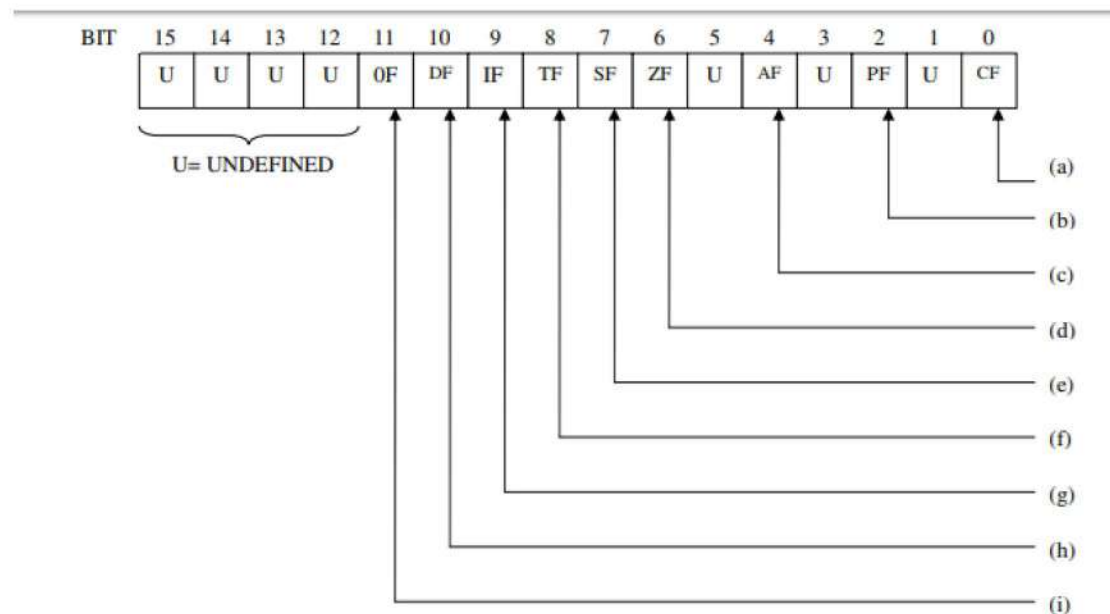
- AX, BX, CX, DX are the general data registers. Low and High bytes of the data registers can be accessed separately
 - AH, BH, CH, DH are the high bytes
 - AL, BL, CL, and DL are the low bytes
- Data Registers are general purpose registers but they also perform special functions
- AX
 - Accumulator Register
 - Preferred register to use in arithmetic, logic and data transfer instructions because it generates the shortest Machine Language Code
 - Must be used in multiplication and division operations
 - Must also be used in I/O operations
- BX
 - Base Register
 - Also serves as an address register
 - Used in array operations
 - Used in Table Lookup operations (XLAT)
- CX
 - Count register
 - Used as a loop counter
 - Used in shift and rotate operations
- DX

- Data register
- Used in multiplication and division
- Also used in I/O operations

Pointer and Index Registers

- Contain the offset addresses of memory locations
- Can also be used in arithmetic and other operations
- SP: Stack pointer
 - Used with SS to access the stack segment
- BP: Base Pointer
 - Primarily used to access data on the stack
 - Can be used to access data in other segments
- SI: Source Index register
 - is required for some string operations
 - When string operations are performed, the SI register points to memory locations in the data segment which is addressed by the DS register. Thus, SI is associated with the DS in string operations.
- DI: Destination Index register
 - It is also required for some string operations.
 - When string operations are performed, the DI register points to memory locations in the data segment which is addressed by the ES register. Thus, DI is associated with the ES in string operations.
- The SI and the DI registers may also be used to access data stored in arrays
- **Flags** is a 16-bit register containing 9 one bit flags.

- **Overflow Flag (OF)** - set if the result is too large positive number, or is too small negative number to fit into destination operand.
- **Direction Flag (DF)** - if set then string manipulation instructions will auto-decrement index registers. If cleared then the index registers will be auto-incremented.
- **Interrupt-enable Flag (IF)** - setting this bit enables makeable interrupts.
- **Single-step Flag (TF)** - if set then single-step interrupt will occur after the next instruction.
- **Sign Flag (SF)** - set if the most significant bit of the result is set.
- **Zero Flag (ZF)** - set if the result is zero.
- **Auxiliary carry Flag (AF)** - set if there was a carry from or borrow to bits 0-3 in the result of previous operation.
- **Parity Flag (PF)** - set if parity (the number of "1" bits) of the result is even.
- **Carry Flag (CF)** - set if there was a carry from or borrow to the most significant bit during last result calculation.



- (a) : CARRY FLAG – SET BY CARRY OUT OF MSB
- (b) : PARITY FLAG – SET IF RESULT HAS EVEN PARITY
- (c) : AUXILIARY CARRY FLAG FOR BCD
- (d) : ZERO FLAG – SET IF RESULT = 0
- (e) : SIGN FLAG = MSB OF RESULT
- (f) : SINGLE STEP TRAP FLAG
- (g) : INTERRUPT ENABLE FLAG
- (h) : STRING DIRECTION FLAG
- (i) : OVERFLOW FLAG

Review Questions

1. Explain the functions of the EU and the BIU.
2. What is pipelining, and how does it make the CPU execute faster?
3. Registers of the 8086 are either _____ bits or _____ bits in length.
4. List the 16-bit registers of the 8086.



CYBER SECURITY TECHNOLOGY ENGINEERING DEPARTMENT

MICROPROCESSOR SECOND STAGE

Lecture.3



Asst. Lect. Haider Saad

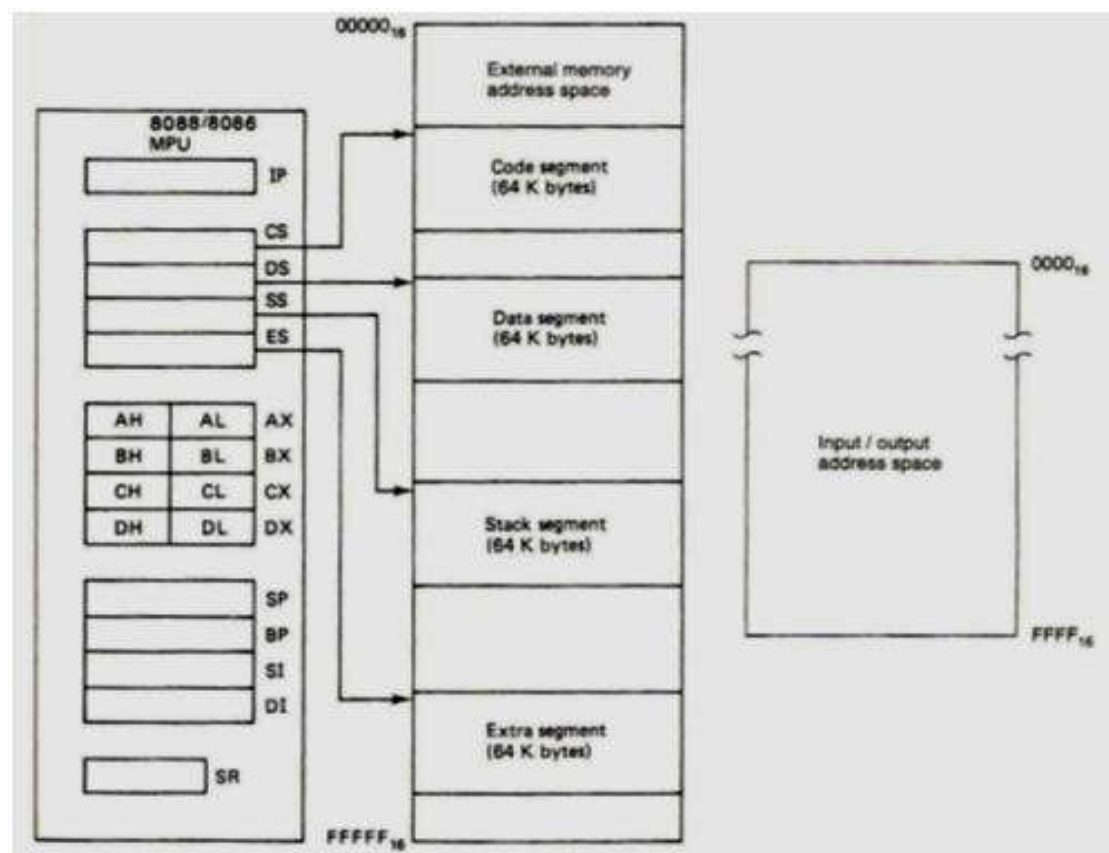
2024 - 2025

Memory Segmentation

Even though the 8086 has a 1Mbyte address space, not all this memory is active at one time. Actually, the 1Mbytes of memory are partitioned into 64Kbyte (65,536) *segments*.

Segment: represents an independently addressable unit of memory consisting of 64K consecutive bytes. Each segment is assigned a *Base Address* that identifies its starting point (identify its lowest address byte-storage location). Only four of these 64Kbyte segments are active a time: the *code segment*, *stack segment*, *data segment*, and *extra segment*.

The addresses of these four segments are held in four segment registers: **CS** (code segment), **SS** (stack segment), **DS** (data segment), and **ES** (extra segment). These registers contain a 16-bit base address that points to the lowest addressed byte of the segment. Note that the segment registers are *user accessible*. This means that the programmer can change their contents through software.



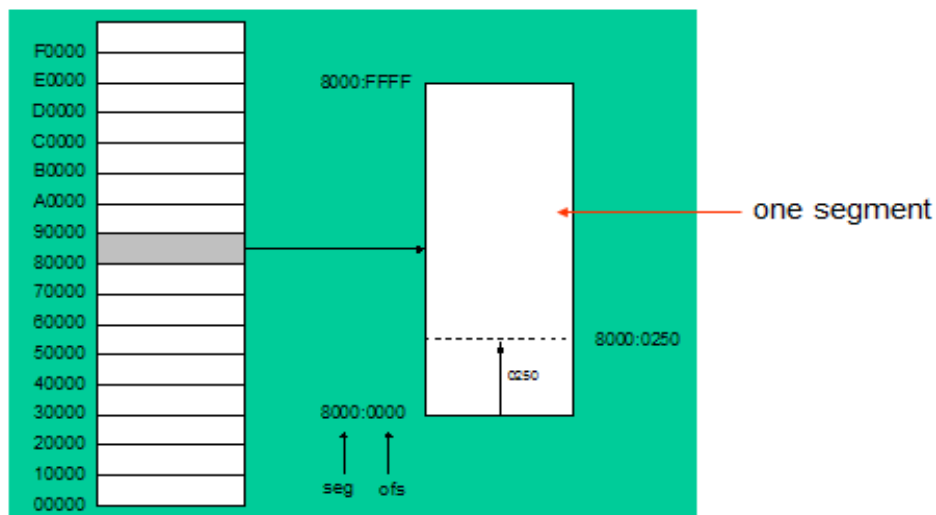
Each memory location address can be represented by one of the following:

1) Logical address is specified as **segment(base):offset**

Where the offset address represents the displacement of the specific location from the segment base (i.e. select a location within 64K bytes). The offset address is 16 bits and stored in one of the μp register as in table below. Each register points to a storage location which is located within specific segment. Each segment starts with offset address 0000H and ending with offset address FFFFH.

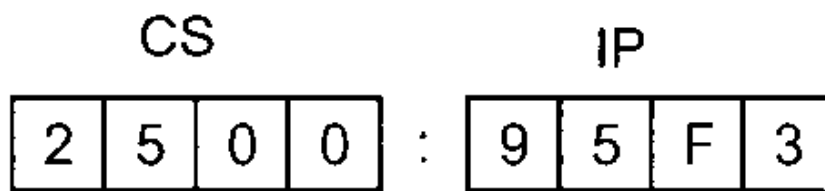
Segment	Offset	Special Purpose
CS	IP	Instruction address
SS	SP or BP	Stack address
DS	BX, DI, SI, an 8- or 16-bit number	Data address
ES	DI for string instructions	String destination address

2) Physical address; it is 20 bits address which is a combination of base and offset address and generated by the BIU.

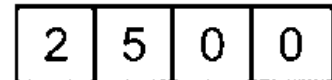


- Physical address is obtained by shifting the segment address 4 bits to the left and adding the offset address

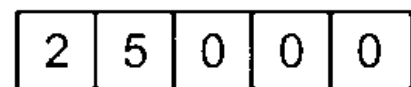
Code segment



1. Start with CS.



2. Shift left CS.



3. Add IP.

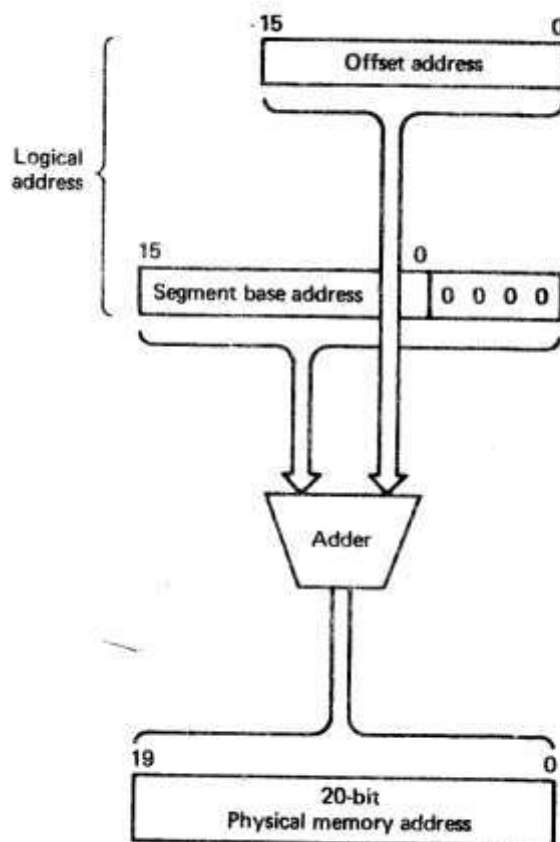
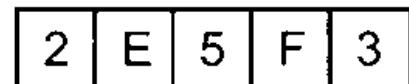


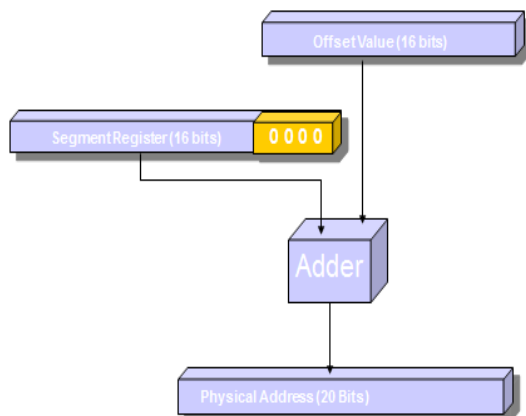
Figure 2.17 Real-mode physical address generation. (Reprinted by permission of Intel Corp. Copyright/Intel 1981)

EX: logical address A4FB:4872

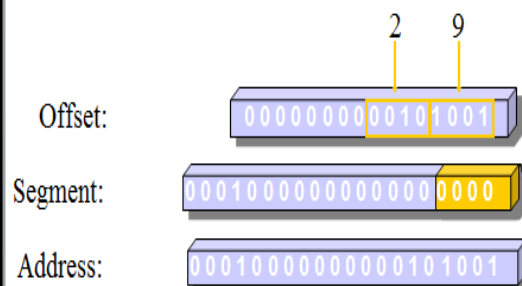
where the base address = A4FB and the offset = 4872

Thus the physical address = A4FB0
 + 4872
 A9822

The BIU has a dedicated adder for determining physical memory addresses



- If the data segment starts at location 1000h and a data reference contains the address 29h where is the actual data?



Example

- If DS=7FA2H and the offset is 438EH

a) Calculate the physical address

$$7FA20 + 438E = 83DAE$$

b) calculate the lower range

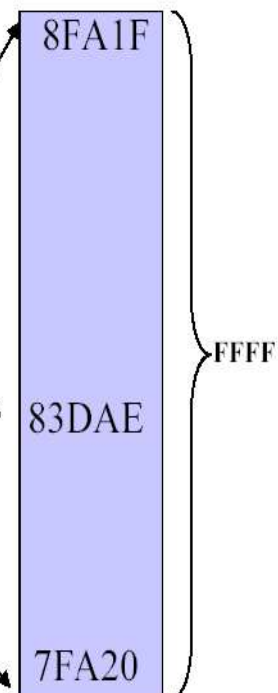
$$7FA20 + 0000 = 7FA20$$

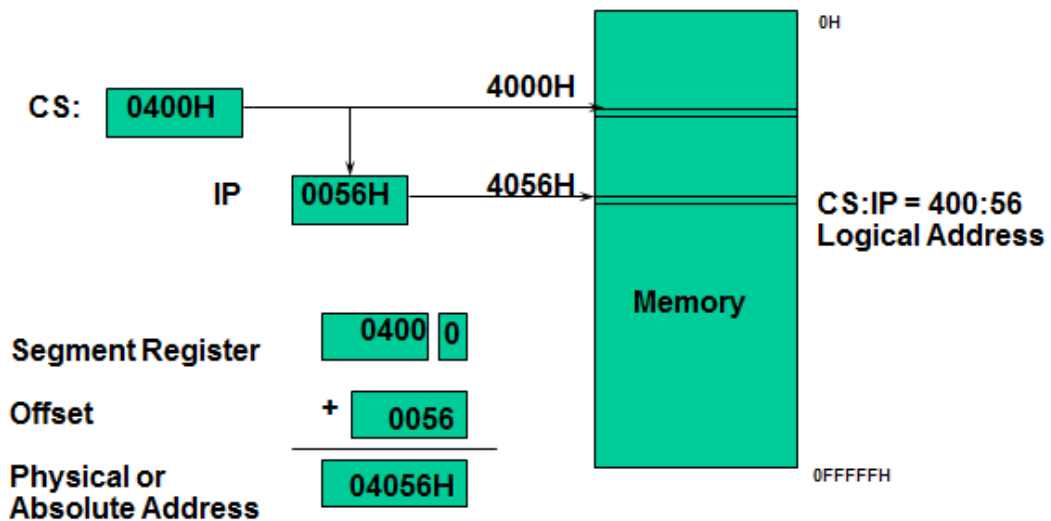
c) Calculate the upper range of the data segment

$$7FA20 + FFFF = 8FA1F$$

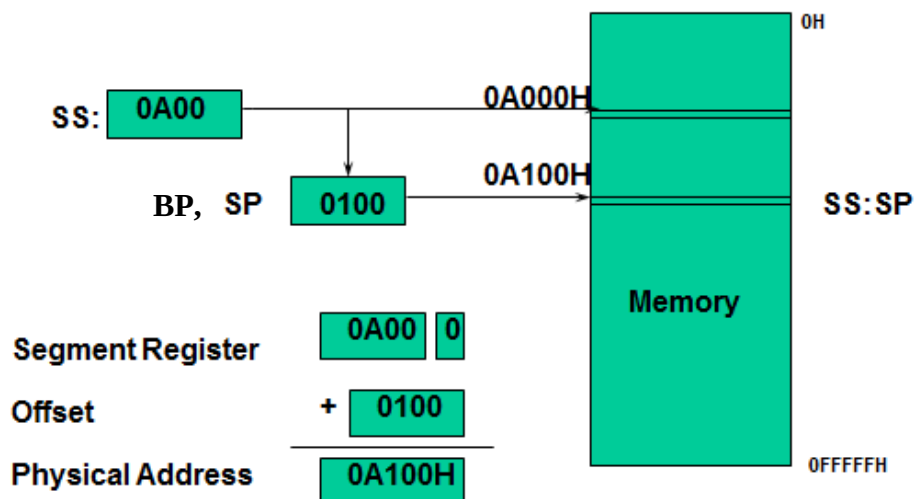
d) Show the logical Address

7FA2:438E

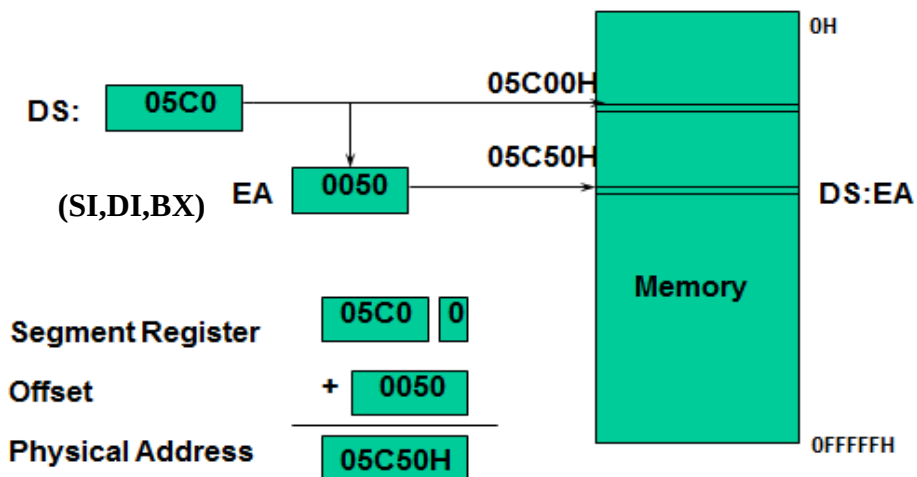




The offset is the distance in bytes from the start of the segment.
 The offset is given by the IP for the Code Segment.
 Instructions are always fetched with using the CS register.



The offset is given by the SP register.
 The stack is always referenced with respect to the stack segment register.

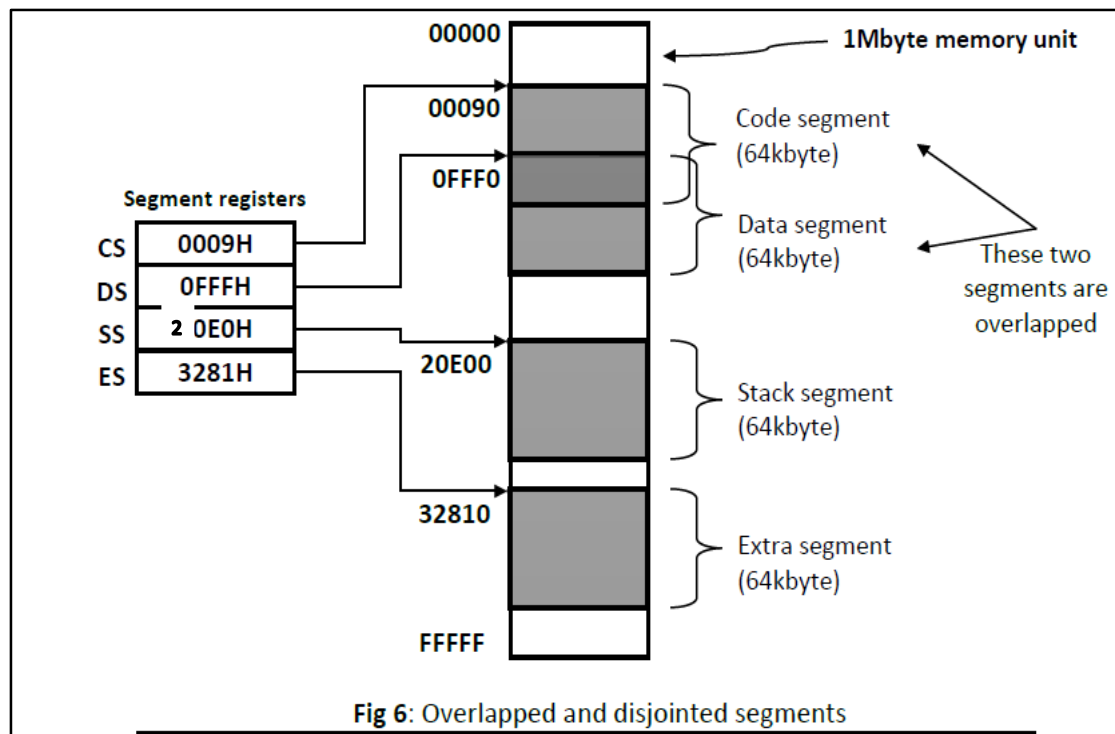


Data is usually fetched with respect to the DS register.
 The effective address (EA) is the offset.
 The EA depends on the addressing mode.

Example 2: Let the segment registers be assigned as follow: CS = 0009H, DS = 0FFFH, SS = 20E0, and ES = 3281H. What will be the lower and the upper range of each segment? Discuss the results

Lower range = base address + 0000H

Upper range = base address + FFFFH



We note here that code segment and data segment are overlapped while other segments are disjointed.

Example 3: if CS = 002AH, and IP = 0023H, write the **logical address** that they represent, then map it to **Physical address**.

Solution:

Logical address = CS:IP
002A : 0023

Physical address = (CS X 10H) + IP = 002A0 + 0023 = 002C3

Example 4: if CS = 002BH, and IP = 0013H, write the **logical address** that they represent, then map it to **Physical address**.

Solution:

Logical address = CS:IP
002B : 0013

Physical address = (CS X 10H) + IP = 002B0 + 0013 = 002C3

Physical addresses are identical here !

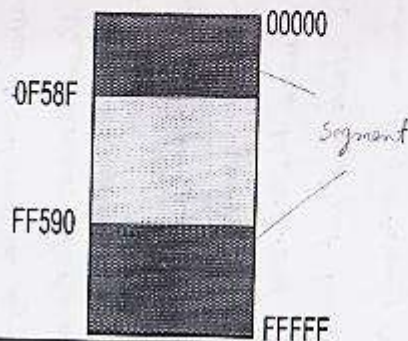
Actually, many different logical addresses map to the same physical address location in memory.

Example 1-9

What is the range of physical addresses if CS = FF59?

Solution:

The low range is FF590 (FF590 + 0000). The range goes to FFFFF and wraps around, from 00000 to 0F58F (FF590 + FFFF = 0F58F), which is illustrated below.



Note that Segment can be wrapped.

Example 1-1

If CS = 24F6H and IP = 634AH, show:

- (a) The logical address
- (b) The offset address
- and calculate:
- (c) The physical address
- (d) The lower range
- (e) The upper range of the code segment

Solution:

- | | |
|--------------------------|--------------------------|
| (a) 24F6:634A | (b) 634A |
| (c) 2B2AA (24F60 + 634A) | (d) 24F60 (24F60 + 0000) |
| (e) 34F5F (24F60 + FFFF) | |

Example 1-3

If DS = 7FA2H and the offset is 438EH,

- | | |
|--|--------------------------------|
| (a) Calculate the physical address. | (b) Calculate the lower range. |
| (c) Calculate the upper range of the data segment. | (d) Show the logical address. |

Solution:

- | | |
|--------------------------|--------------------------|
| (a) 83DAE (7FA20 + 438E) | (b) 7FA20 (7FA20 + 0000) |
| (c) 8FA1F (7FA20 + FFFF) | (d) 7FA2:438E |

Example 1-4

Assume that the DS register is 578C. To access a given byte of data at physical memory location 67F66, does the data segment cover the range where the data is located? If not, what changes need to be made?

Solution:

No, since the range is 578C0 to 678BF, location 67F66 is not included in this range. To access that byte, DS must be changed so that its range will include that byte.

H/W: IF CS=0192, ES=6786, DS=E224 AND CS=86B0, Where 712A5 is located ?

H/W; If a physical address 1B233 is located at code segment and the CS=1920, What will be the value of IP register.



CYBER SECURITY TECHNOLOGY ENGINEERING DEPARTMENT

MICROPROCESSOR SECOND STAGE

Lecture.4,5



Asst. Lect. Haider Saad

2024 - 2025

Addressing Modes

- When 8086 execute an instruction, it performs the specified function on data called operand.
- Operand may be a part of the instruction or may be stored in internal register, M.L. or it may be held at an I/O port.
- 8086 MP provided with various addressing modes which is a method of specifying and access operand.

1- Register Addressing Mode

Register addressing refers to operands that contained in one of the registers. The instruction of this form often execute very quickly, because they don't require any memory accesses beyond initial instruction fetch cycles.

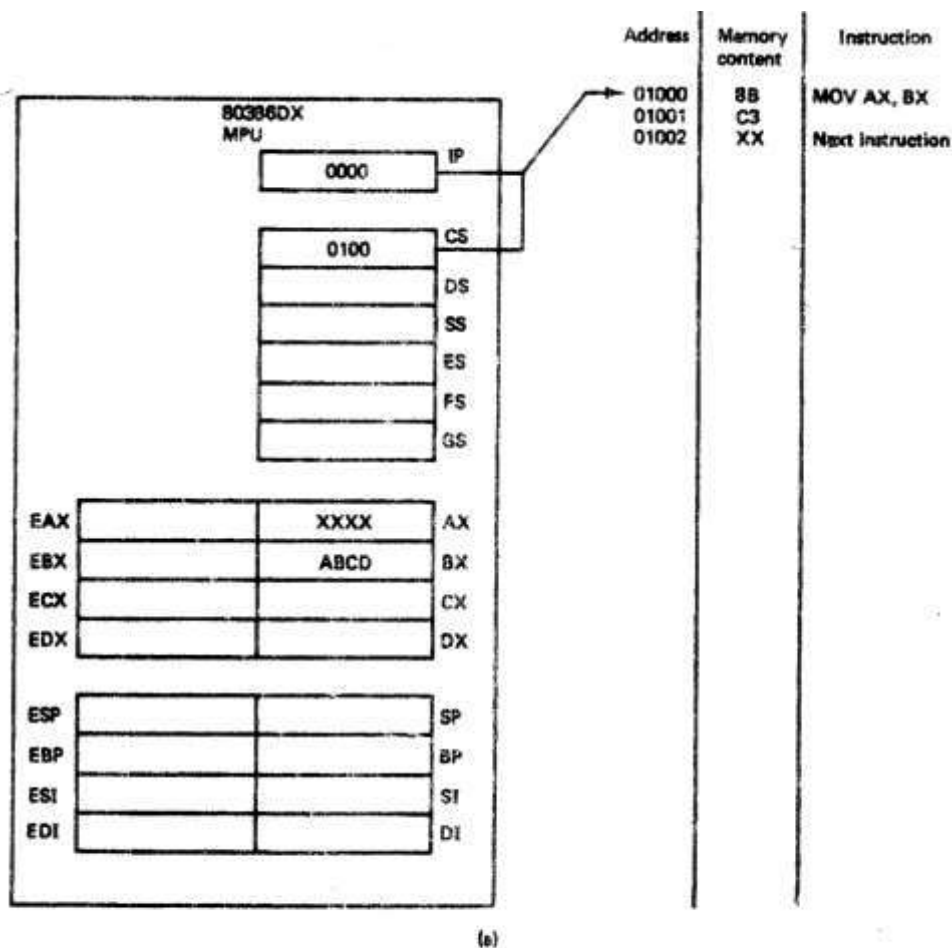


Figure 3.8 (a) Register addressing mode instruction before fetch and execution. (b) After execution.

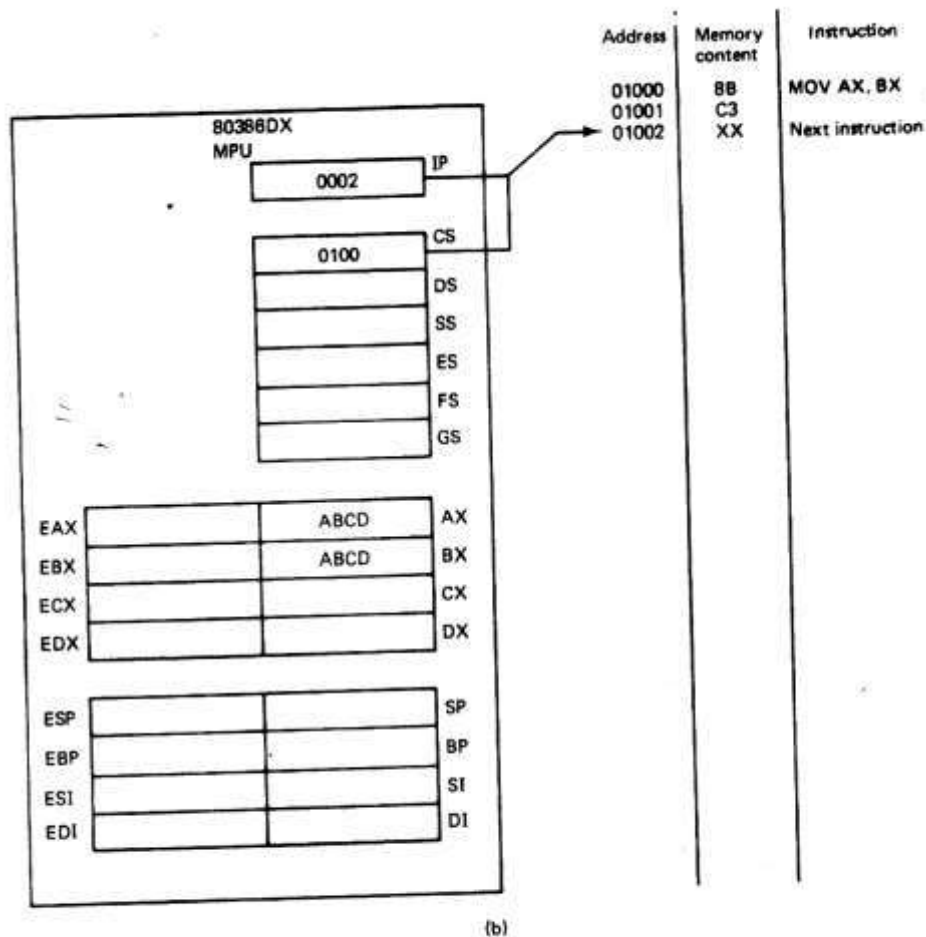


Figure 3.8 (Continued)

Ex. *MOV DS, AX* → 8E D8

DEC DX → 4A

Ex.: If AX=1000H & BL =80H, what will be the value of AX after executing *MOV AL, BL*

Ans.

AX=1080 H

2- Immediate Addressing Mode

Immediate data represents a constant that we wish to place somewhere, by moving it into a register or a memory location

MOV AX, 1000H → B8 00 10

MOV AL, 7 → B0 07

****Note** that the immediate data are coded directly into the instruction (the lower byte followed by the upper byte).

****Note** the assembler recognizes the 07 in the second instruction as one-byte data because the size of the first operand (AL) is one byte.

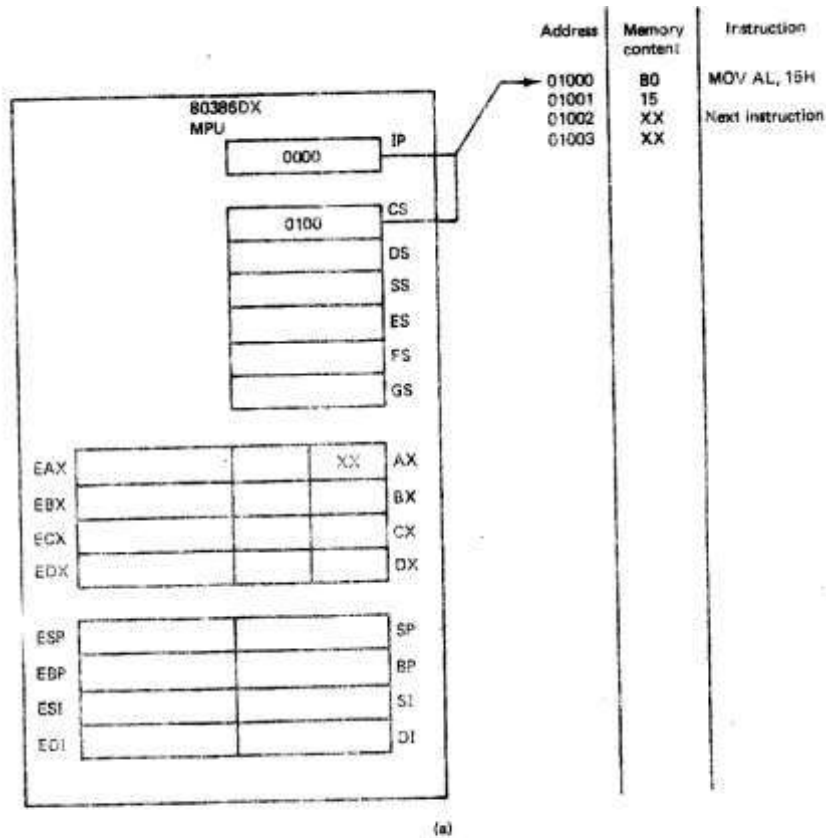


Figure 3.10 (a) Immediate addressing mode instruction before fetch and execution. (b) After execution.

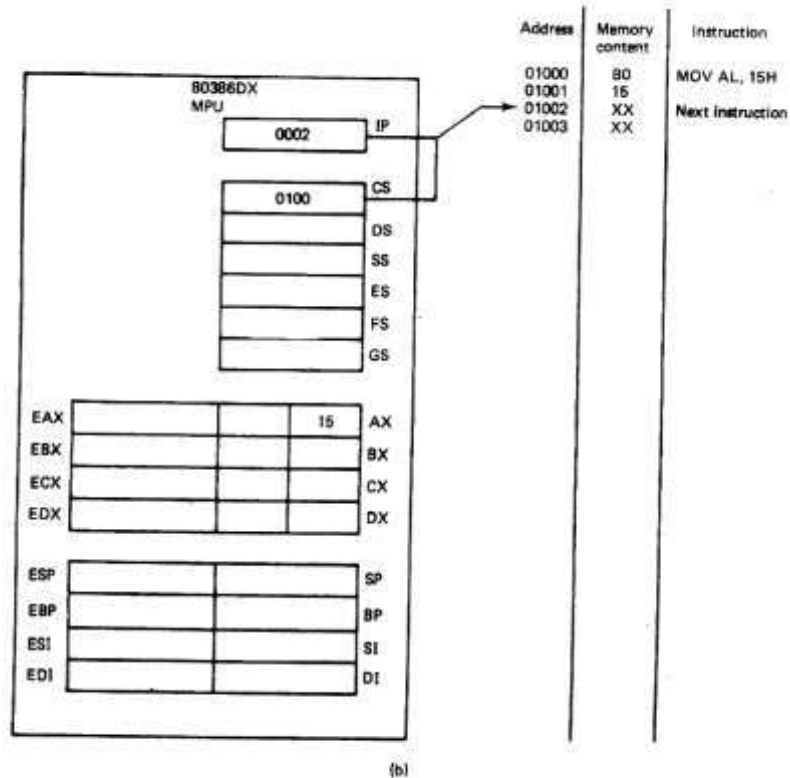


Figure 3.10 (Continued)

Ex.: write the code and execution results after for the following instructions if AX=2345 H.

1 - $MOV\ AL.7 \xrightarrow{\text{code}} B0\ 07 \xrightarrow{\text{result}} AX = 2307\ H$

2 - $MOV\ AX.7 \xrightarrow{\text{code}} B8\ 07\ 00 \xrightarrow{\text{result}} AX = 0007H$

3- Direct Addressing Mode

In this addressing mode effective address is coded directly into the instruction. Ds reg. is used usually whenever [] are used in the operand field.

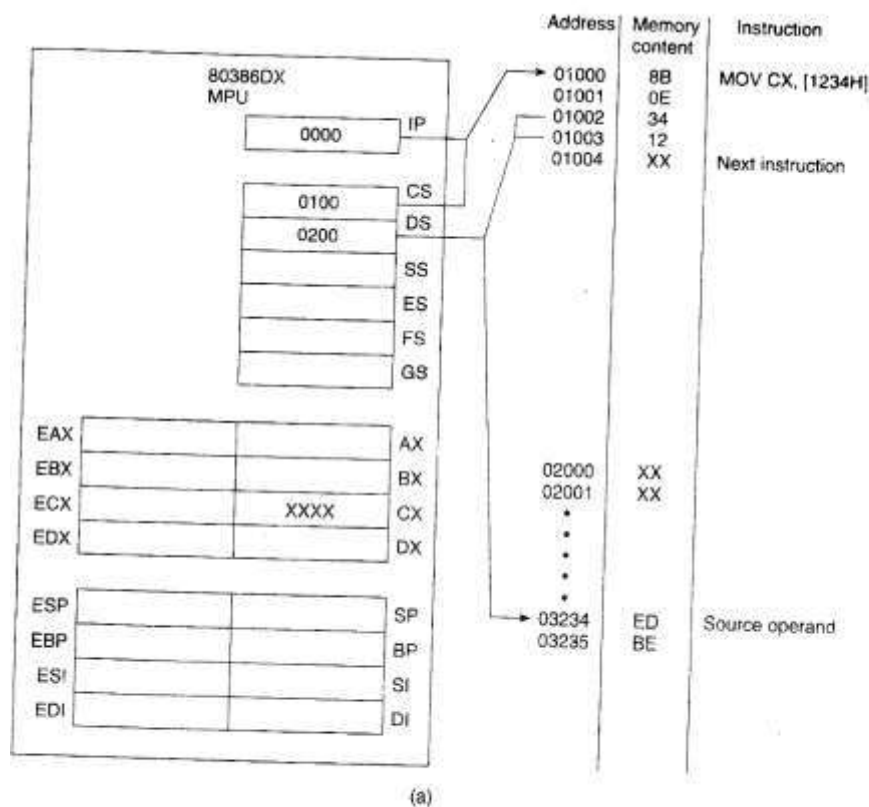


Figure 3.13 (a) Direct addressing mode instruction before fetch and execution, (b) After execution.

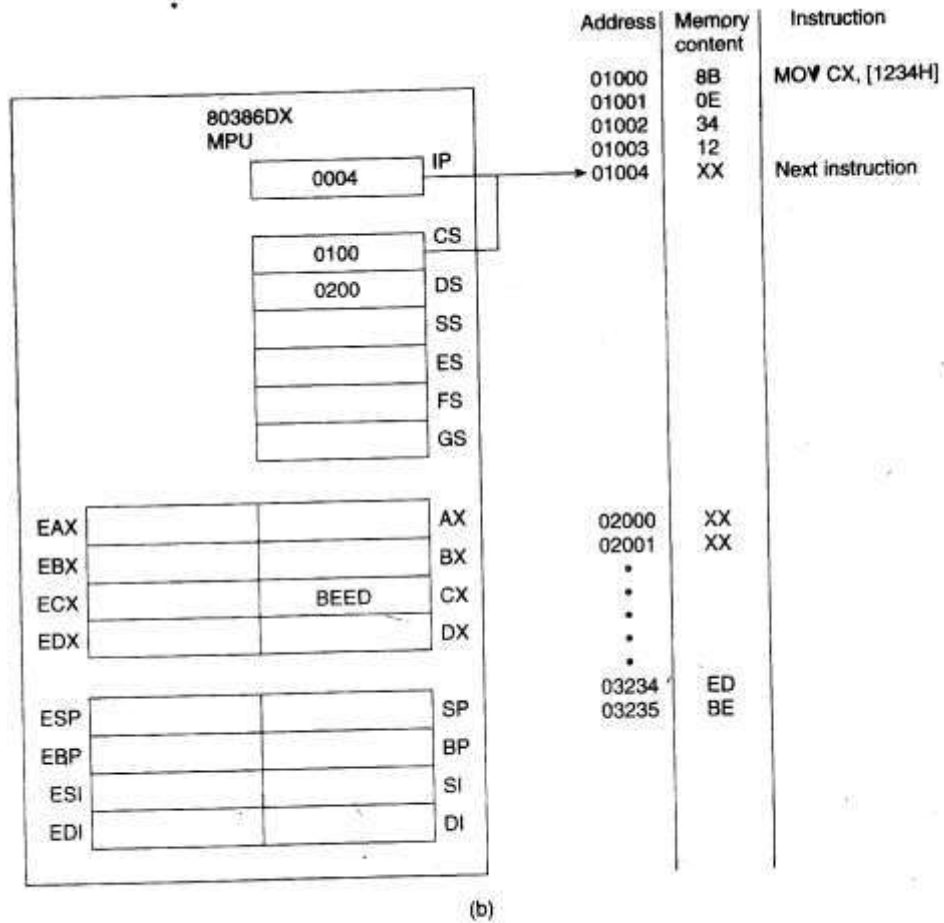
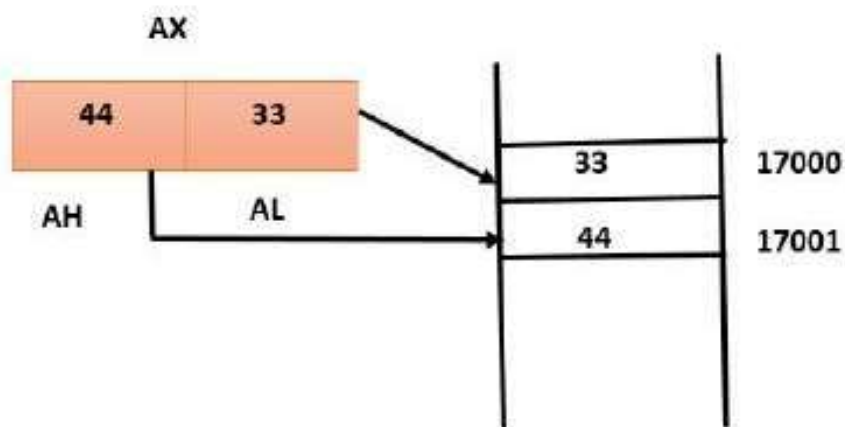


Figure 3.13 (Continued)

Ex.: MOV [7000], AX AX= 4433 /DS=1000
 Physical 10000
 Address 7000 +
 17000 H



Example 1-15

Find the physical address of the memory location and its contents after the execution of the following, assuming that DS = 1512H.

```
MOV    AL,99H
MOV    [3518],AL
```

Solution:

First AL is initialized to 99H, then in line two, the contents of AL are moved to logical address DS:3518 which is 1512:3518. Shifting DS left and adding it to the offset gives the physical address of 18638H ($15120H + 3518H = 18638H$). That means after the execution of the second instruction, the memory location with address 18638H will contain the value 99H.

4- Register Indirect Addressing Mode

In this addressing mode, one of 4 register can be used within the square brackets to specify a memory location BX, BP, SI, and DI.

The value (16 bit) of BX, SI & DI are added to DS reg.
and the value (16 bit) of BP is added to SS reg.

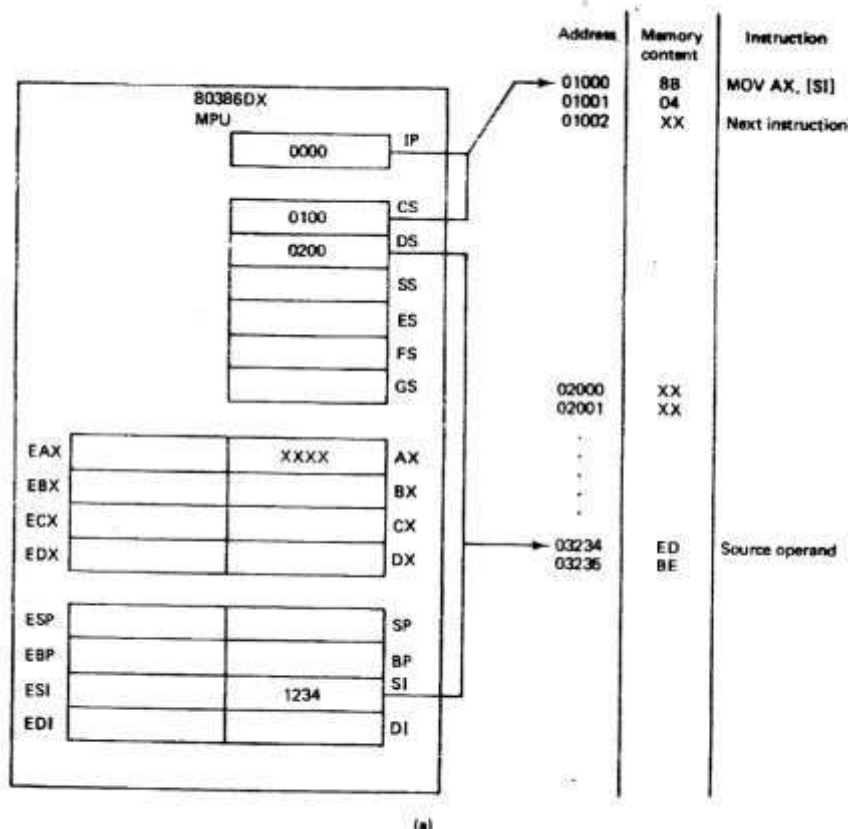


Figure 3.15 (a) Instruction using register indirect addressing mode before fetch and execution. (b) After execution.

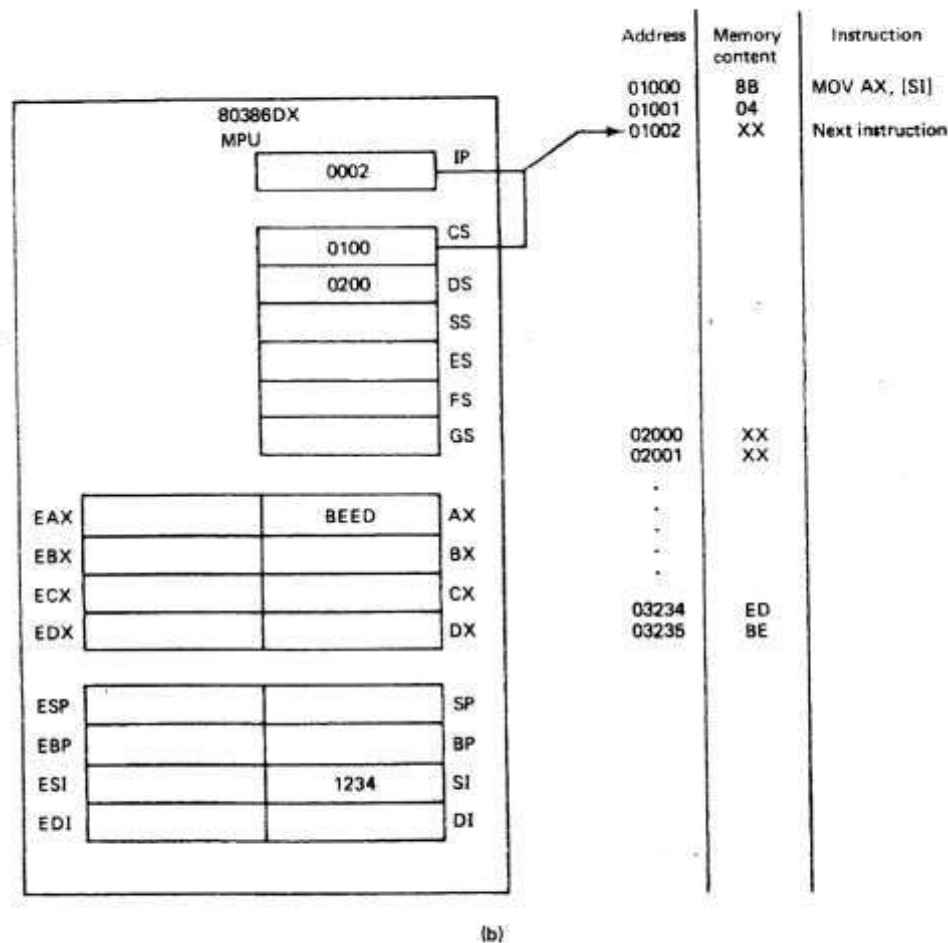


Figure 3.15 (Continued)

Ex.: what will be the result after executing the following instructions, if DS reg.= 800H, SS reg. =600H, SI reg. =2000H, and BP reg. =32 H,
MOV [SI],DL
MOV [BP],DH

Ans.

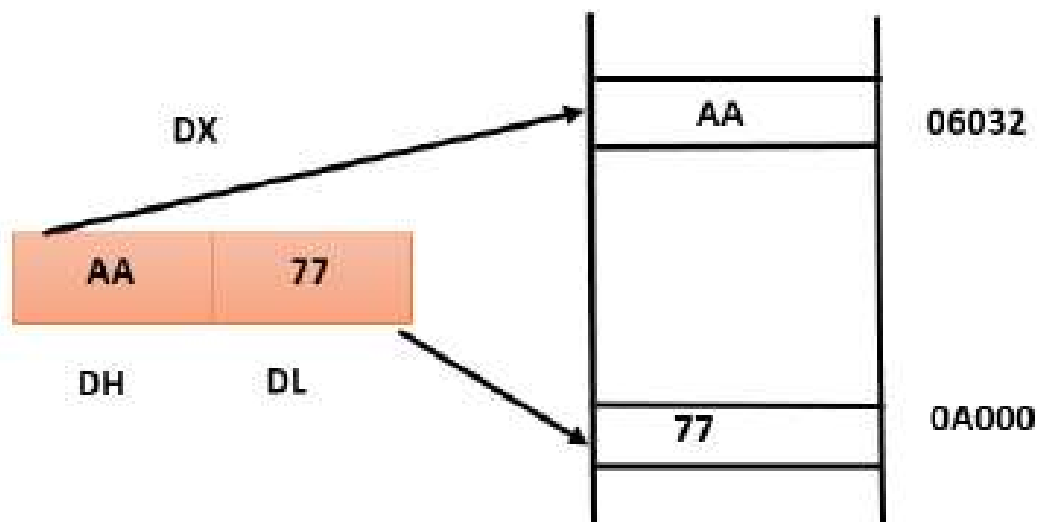
$$DS:SI \rightarrow 800:2000 \rightarrow \text{effective address} = \begin{matrix} 08000 \\ 2000 + \\ 0A000 \end{matrix}$$

In data segment

$$SS:BP \rightarrow 600:0032 \rightarrow EA = \begin{matrix} 06000 \\ 0032 + \\ 06032 \end{matrix}$$

In stack segment.

$$\begin{cases} MOV A.M \\ STAX D \end{cases} = 8085$$



Example 1-16

Assume that $DS = 1120$, $SI = 2498$, and $AX = 17FE$. Show the contents of memory locations after the execution of

`MOV [SI],AX`

Solution:

The contents of AX are moved into memory locations with logical address $DS:SI$ and $DS:SI + 1$; therefore, the physical address starts at DS (shifted left) + $SI = 13698$. According to the little endian convention, low address $13698H$ contains FE , the low byte, and high address $13699H$ will contain 17 , the high byte.

5- Based Addressing Mode

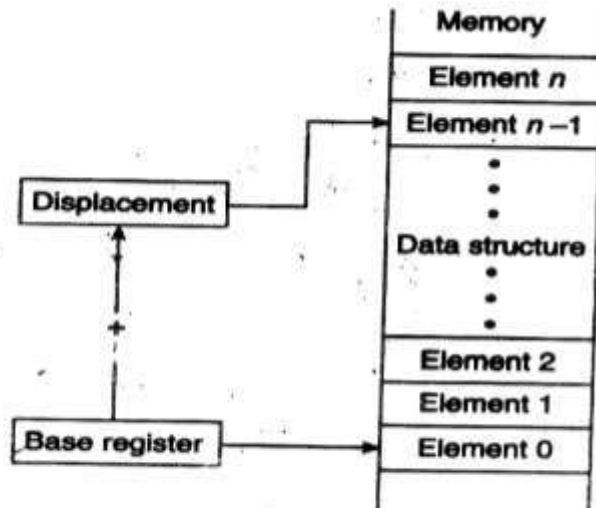
This addressing mode uses one of two base reg. (**BX** or **BP**) it is similar to register indirect except an 8 bit or 16 bit displacement may be included in operand field.

****Note:** the displacement is interpreted as a signed, 2's complement no., then 8 bit displacement gives the range $-128 \rightarrow +127$

And 16 bit displacement gives the range $-32768 \rightarrow +32767$

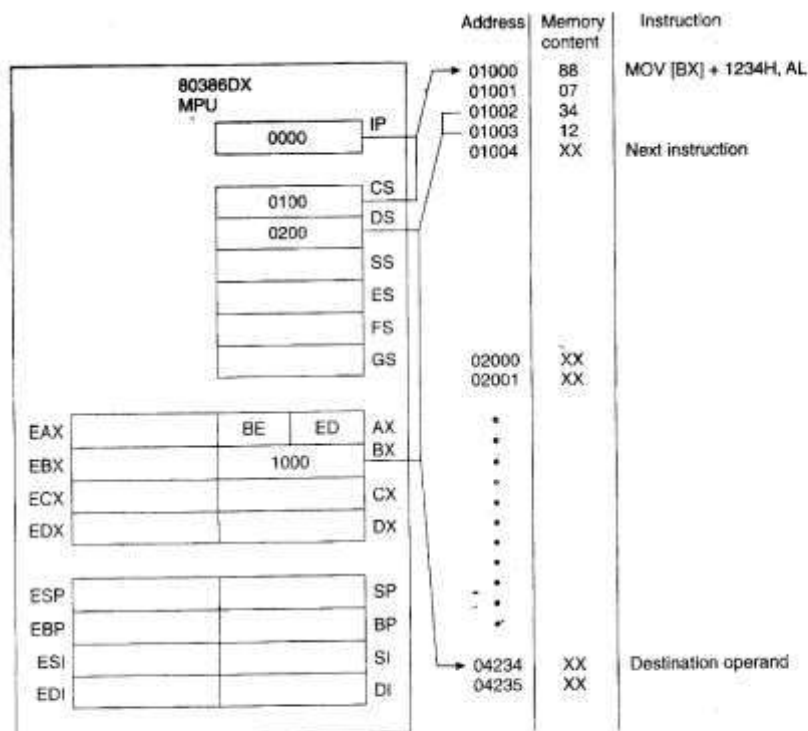
$$PA = \left\{ \begin{matrix} CS \\ DS \\ SS \\ ES \\ FS \\ GS \end{matrix} \right\} : \left\{ \begin{matrix} BX \\ BP \end{matrix} \right\} + \left\{ \begin{matrix} 8\text{-bit displacement} \\ 16\text{-bit displacement} \end{matrix} \right\}$$

(a)



(b)

Figure 3.16 (a) Computation of a based address. (b) Based addressing of a structure of data.



(a)

Figure 3.17 (a) Instruction using direct base pointer addressing mode before fetch and execution. (b) After execution.

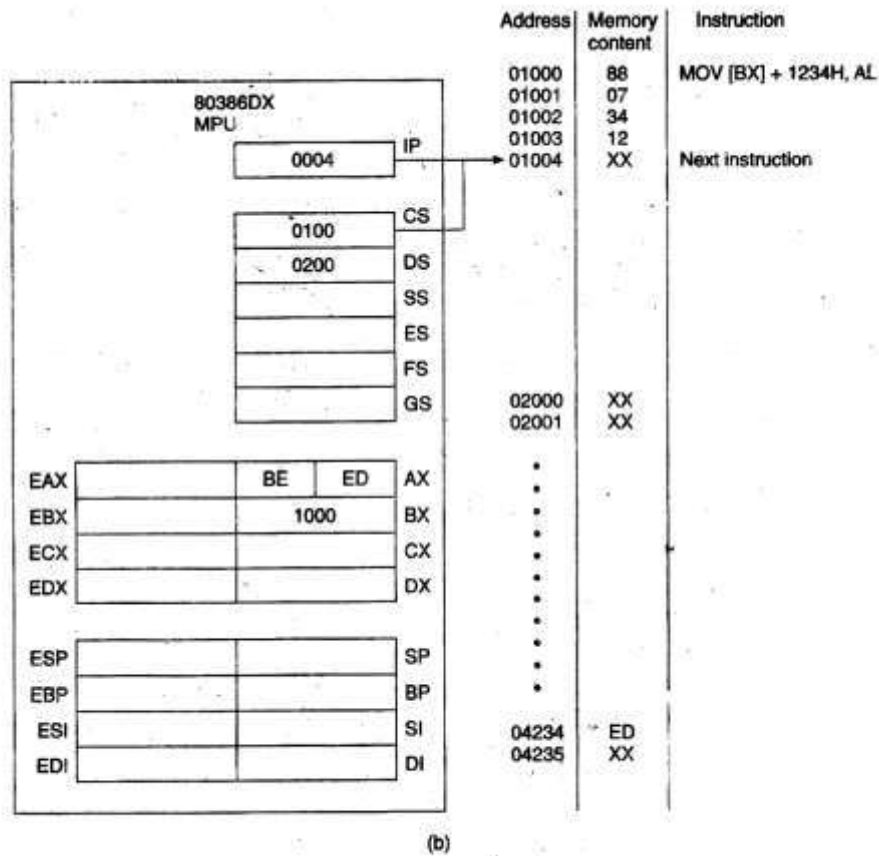
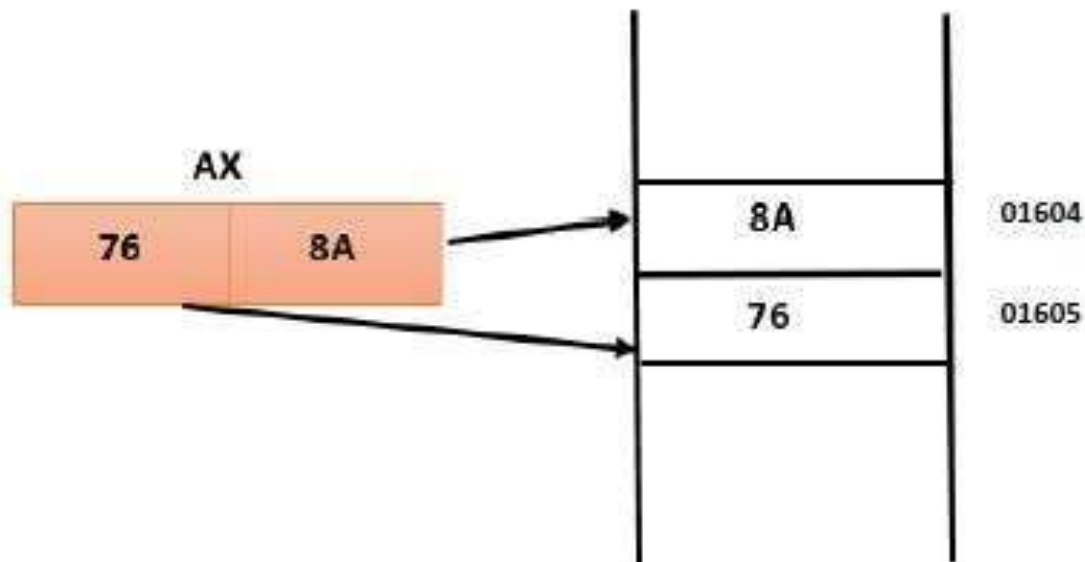


Figure 3.17 (Continued)

Ex.: if BX= 600H, DS=100H, AX=768AH, what will be the result after executing MOV[BX+4], AX.

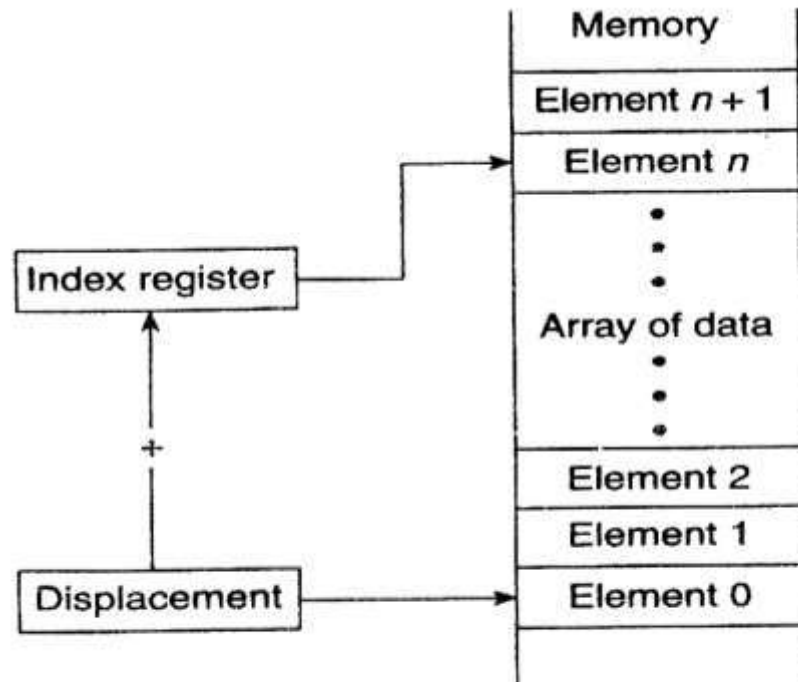
Ans.:

$$EA \text{ is data seg.} = \begin{array}{r} 01000 \\ 600 + \\ \hline 01600 \\ 4 + \\ \hline 01604 \end{array}$$



6- Indexed Addressing Mode

Like based addressing mode except that we must use (SI or DI) in the operand field.



(a)

PA = Segment base: index + displacement

$$PA = \left\{ \begin{array}{c} CS \\ DS \\ SS \\ ES \\ FS \\ GS \end{array} \right\} : \left\{ \begin{array}{c} SI \\ DI \end{array} \right\} + \left\{ \begin{array}{c} 8\text{-bit displacement} \\ 16\text{-bit displacement} \end{array} \right\}$$

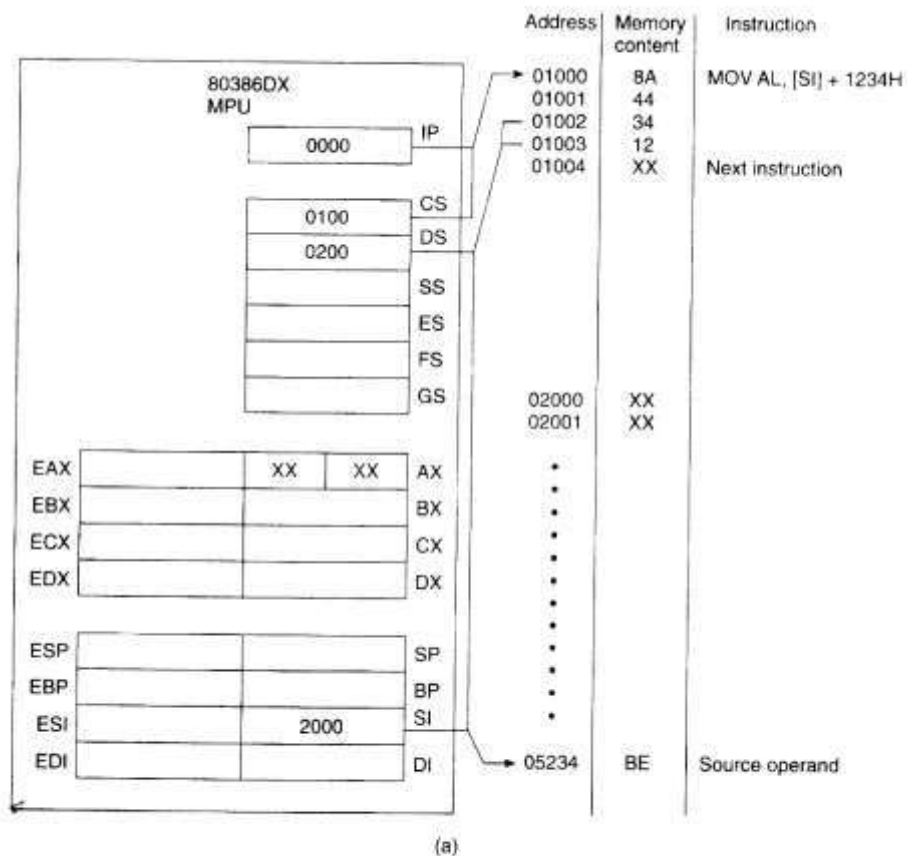


Figure 3.19 (a) Instruction using direct indexed addressing mode before fetch and execution. (b) After execution.

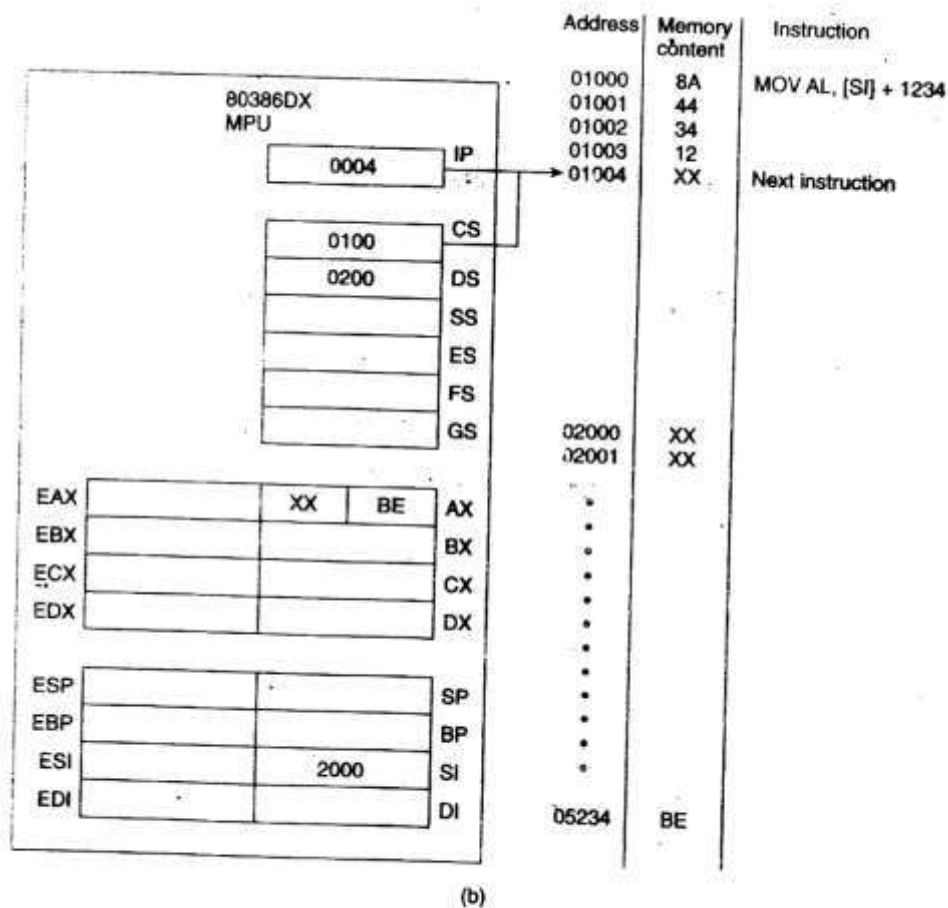


Figure 3.19 (Continued)

Ex.: DS=200H, DI=30H, BX=8899H, MOV[DI-9], BL.

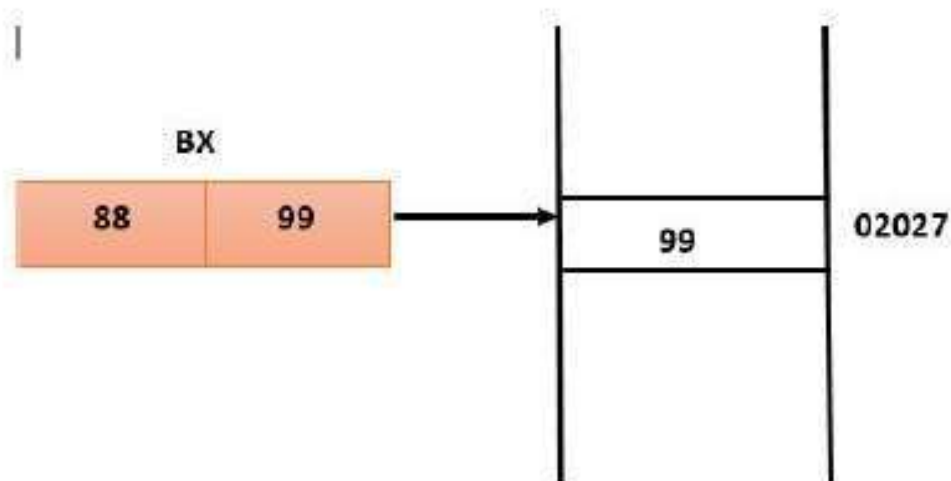
MOV [DI + 2's comp. (9)]. BL

```

    02000
      30
EA = 02030
      9 -
    02027
  
```

```

0000 0010 0000 0011 0000
1111 1111 1111 1111 0111 +
0000 0010 0000 0010 0111
  0      2      0      2      7
  
```



Example 1-17

Assume that DS = 4500, SS = 2000, BX = 2100, SI = 1486, DI = 8500, BP = 7814, and AX = 2512. Show the exact physical memory location where AX is stored in each of the following. All values are in hex.

- (a) MOV [BX]+20,AX (b) MOV [SI]+10,AX
(c) MOV [DI]+4,AX (d) MOV [BP]+12,AX

Solution:

In each case PA = segment register (shifted left) + offset register + displacement.

- (a) DS:BX+20 location 47120 = (12) and 47121 = (25)
(b) DS:SI+10 location 46496 = (12) and 46497 = (25)
(c) DS:DI+4 location 4D504 = (12) and 4D505 = (25)
(d) SS:BP+12 location 27826 = (12) and 27827 = (25)

7- Based Indexed Addressing Mode

This addressing mode combine the feature of based and indexed addressing

Ex.:

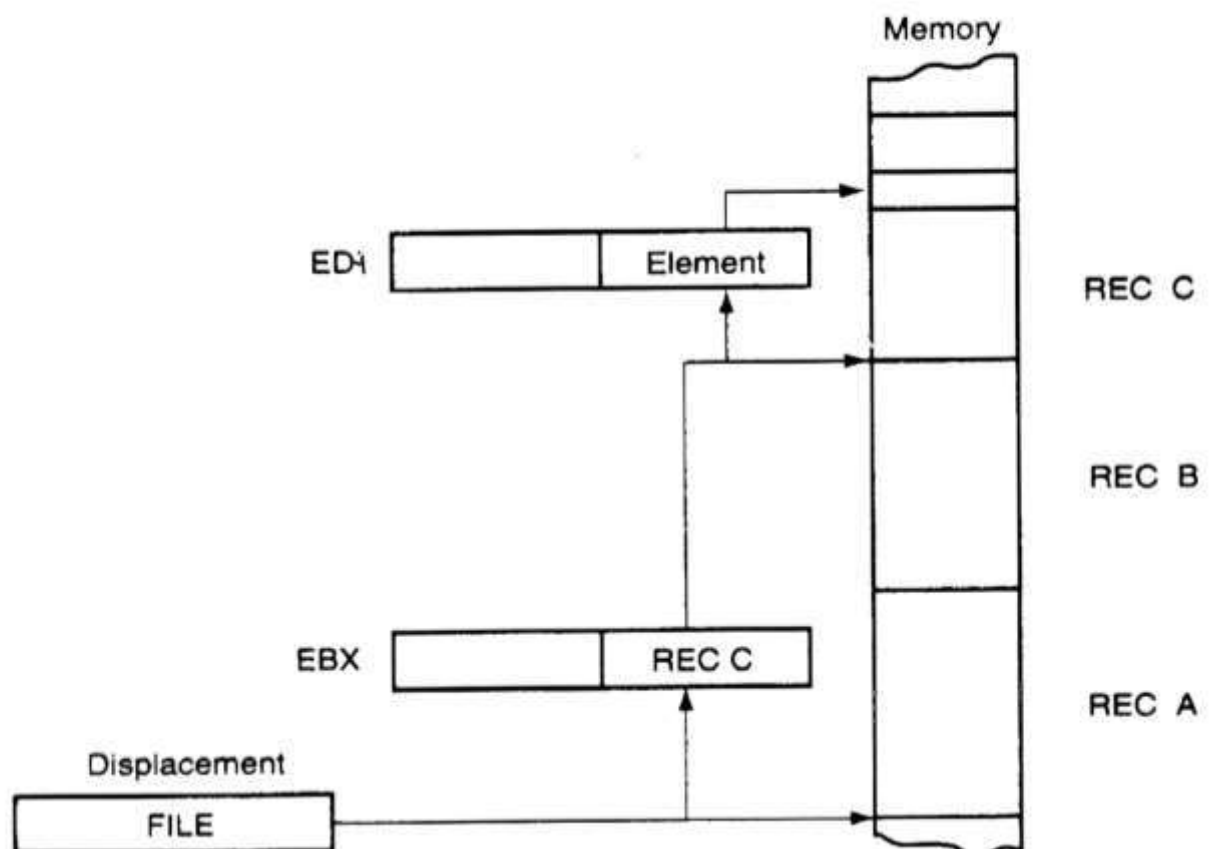
$MOV AL.[BP + SI]$
 $MOV AL.[BP + DI]$ } using BP relative to SS reg.

$MOV AL.[BX + SI]$
 $MOV AL.[BX + DI]$ } using BX relative to DS reg.

PA = Segment base: base + index + displacement

$$PA = \left\{ \begin{matrix} CS \\ DS \\ SS \\ ES \\ FS \\ GS \end{matrix} \right\} : \left\{ \begin{matrix} BX \\ BP \end{matrix} \right\} + \left\{ \begin{matrix} SI \\ DI \end{matrix} \right\} + \left\{ \begin{matrix} 8\text{-bit displacement} \\ 16\text{-bit displacement} \end{matrix} \right\}$$

(b)



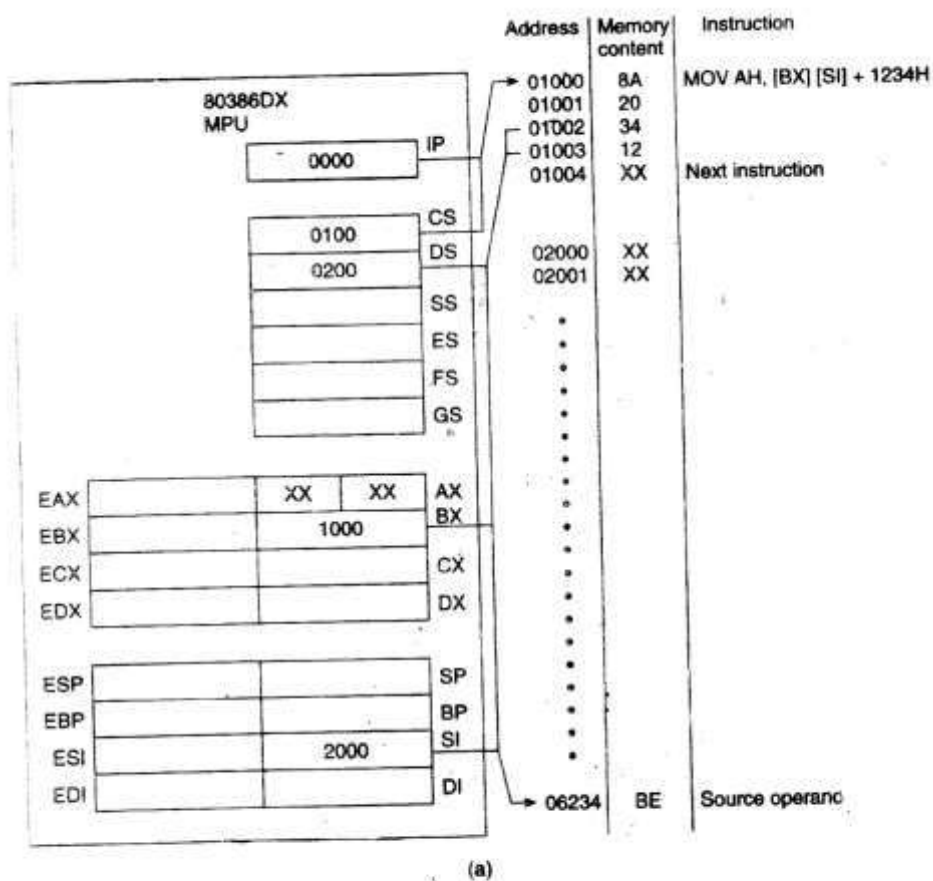


Figure 3.21 (a) Instruction using based-indexed addressing mode before fetch and execution. (b) After execution.

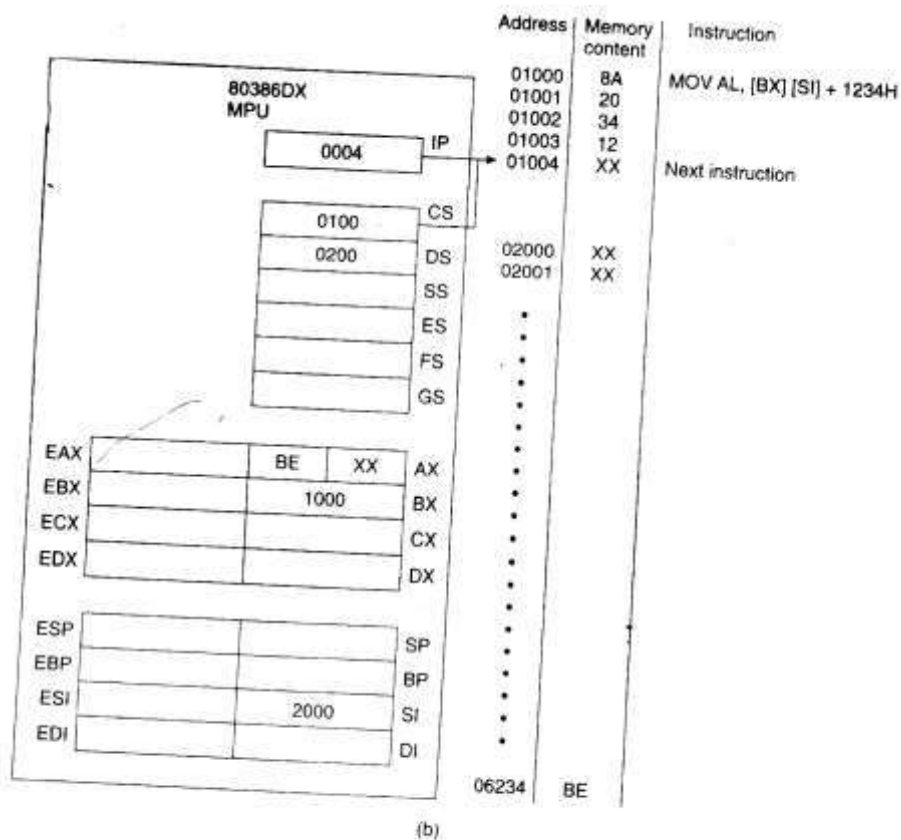
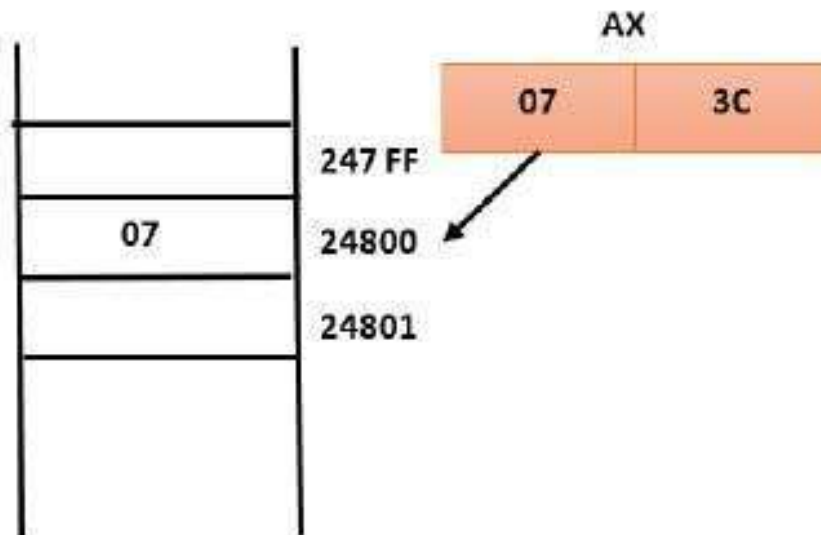


Figure 3.21 (Continued)

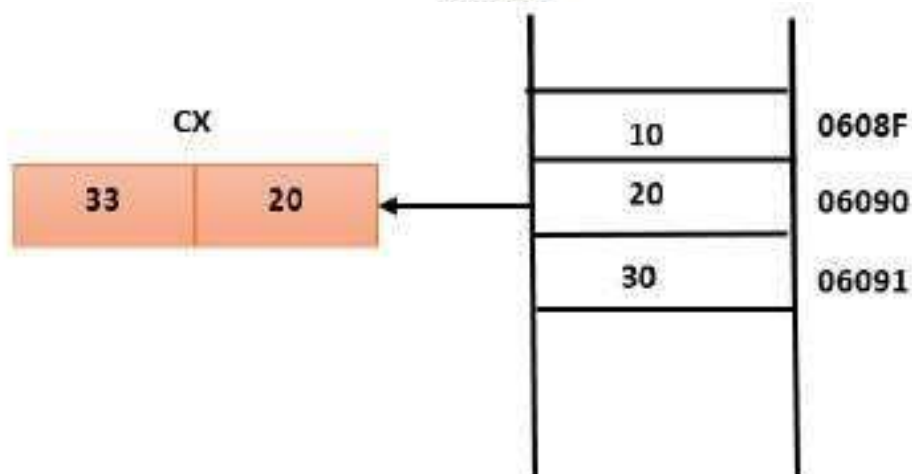
Ex.: what is the effective address generated by MOV [BP+SI], AH, if SS reg.=2000 H , BP reg.=4000 H, and SI reg. =800H.

$$EA = \begin{array}{r} 20000 \\ 04000 + \\ 00800 + \\ \hline 24800 \end{array}$$



Ex.: what is memory location accessed by MOV CL, [BX+DI+2080H], assume that DS reg.=300H, BX=1000H & DI reg.=10H

$$EA = \begin{array}{r} 03000 \\ 1000 + \\ 00010 + \\ 2080 + \\ \hline 06090 \end{array}$$



****Note:** this addressing give the programmer the option of using 2 reg. and the addition displacement makes the addressing mode more flexible.

8- String Addressing Mode

This mode is used with string instruction (MOVS, MOVB,...) the source operand is represented by **DS:SI** & the destination operand is represented **ES:DI**

****Both SI & DI don't appear in the instruction but they are used during the execution and updated automatically.**

9- Port Address Model

For memory mapped ports, any of the memory addressing mode can be used for port address

For I/O address space, only the direct addressing mode and indirect addressing mode using DX is available.

Ex.: IN AL, 15H

Ex.: IN AL,DX

Table 1-4: Sample Segment Overrides

Instruction	Segment Used	Default Segment
MOV AX,CS:[BP]	CS:BP	SS:BP
MOV DX,SS:[SI]	SS:SI	DS:SI
MOV AX,DS:[BP]	DS:BP	SS:BP
MOV CX,ES:[BX]+12	ES:BX+12	DS:BX+12
MOV SS:[BX][DI]+32,AX	SS:BX+DI+32	DS:BX+DI+32

Table 1-5: Summary of 80x86 Addressing Modes

Addressing Mode	Operand	Default Segment
Register	reg	none
Immediate	data	none
Direct	[offset]	DS
Register indirect	[BX]	DS
	[SI]	DS
	[DI]	DS
Based relative	[BX]+disp	DS
	[BP]+disp	SS
Indexed relative	[DI]+disp	DS
	[SI]+disp	DS
Based indexed relative	[BX][SI]+disp	DS
	[BX][DI]+disp	DS
	[BP][SI]+ disp	SS
	[BP][DI]+ disp	SS



CYBER SECURITY TECHNOLOGY ENGINEERING DEPARTMENT

MICROPROCESSOR SECOND STAGE

Lecture.6



Asst. Lect. Haider Saad

2024 - 2025

Memory address space and data organization

8086 can supports 1Mbyte of external memory that organized as individual bytes of data stored at consecutive addresses over the address range 00000_{16} to $FFFFF_{16}$. The 8086 can access any two consecutive bytes as a word of data. The lower-addressed byte is the least significant byte of the word, and the higher- addressed byte is its most significant byte.

Example 1: For the 1Mbyte memory shown in Fig 2, storage location of address 00009_{16} contains the value $00000111_2 = 7_{16}$, while the location of address 00010_{16} contains the value $01111101 = 7D_{16}$. The 16-bit word **225A**₁₆ is stored in the locations $0000C_{16}$ to $0000D_{16}$.

00009	07
0000A	
0000B	
0000C	5A
0000D	22
0000E	
0000F	
00010	7D

Fig2:Part of 1Mbyte memory

The word of data is at an even-address boundary if its least significant byte is in even address. It's also called aligned word. The word of data is at an odd-address boundary if its least significant byte is in odd address. It's also called misaligned word, as shown in Fig 3. To store double word four locations are needed. The double word that it's least significant byte store at an address that is a multiple of 4 (e.g. 0, 4, 8,...) as shown in Fig 4.

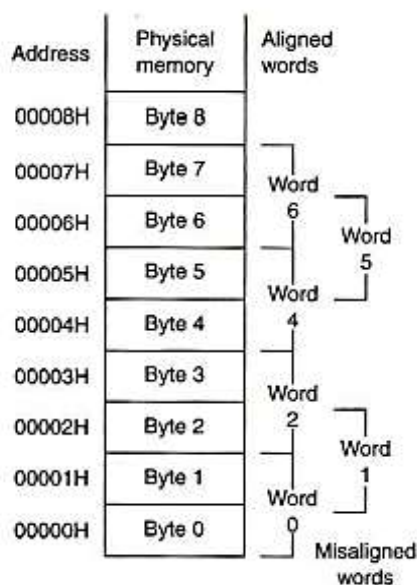


Fig 3 Aligned and misaligned word

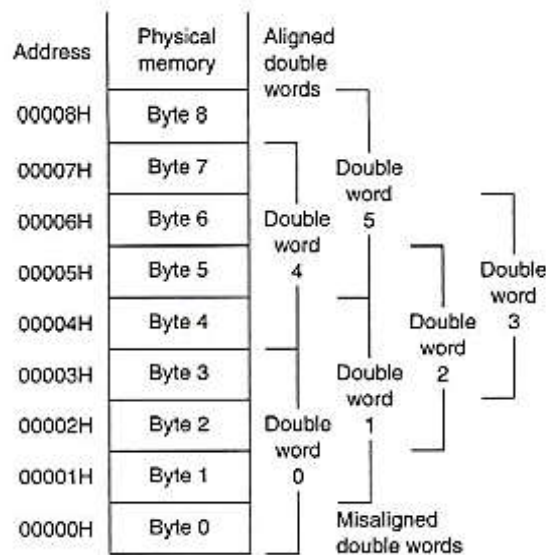


Fig 4 Aligned and misaligned double word

Hardware Organization of the Memory Address Space

The 8088 μ P's memory address space as shown in figure below is implemented as single 1 Mbyte memory bank. The 8086 μ P's 1 Mbyte memory address space as shown in figure below is implemented as two independent 512 Kbyte banks:

- ♣ Data bytes associated with an even address (00000, 00002, , FFFFE) reside in the low bank.

- ♣ Data bytes associated with an odd addresses (00001, 00003, , FFFFF) reside in the high bank.

Address bits A1 through A19 select the location that is to be accessed. They are applied to both banks in parallel. A0 (BLE) and bank high enable ($\overline{\text{BHE}}$) are used as bank-select signals:

- ♣ A0 = 0 causes the low bank to be enabled. While $\overline{\text{BHE}}$ = 0 causes the high bank to be enabled. Each of the memory banks provides half of the 8086 μ P's 16-bit data bus. The lower bank transfers bytes of data over data lines D0 through D7, while data transfers for a high bank use D8 through D15.

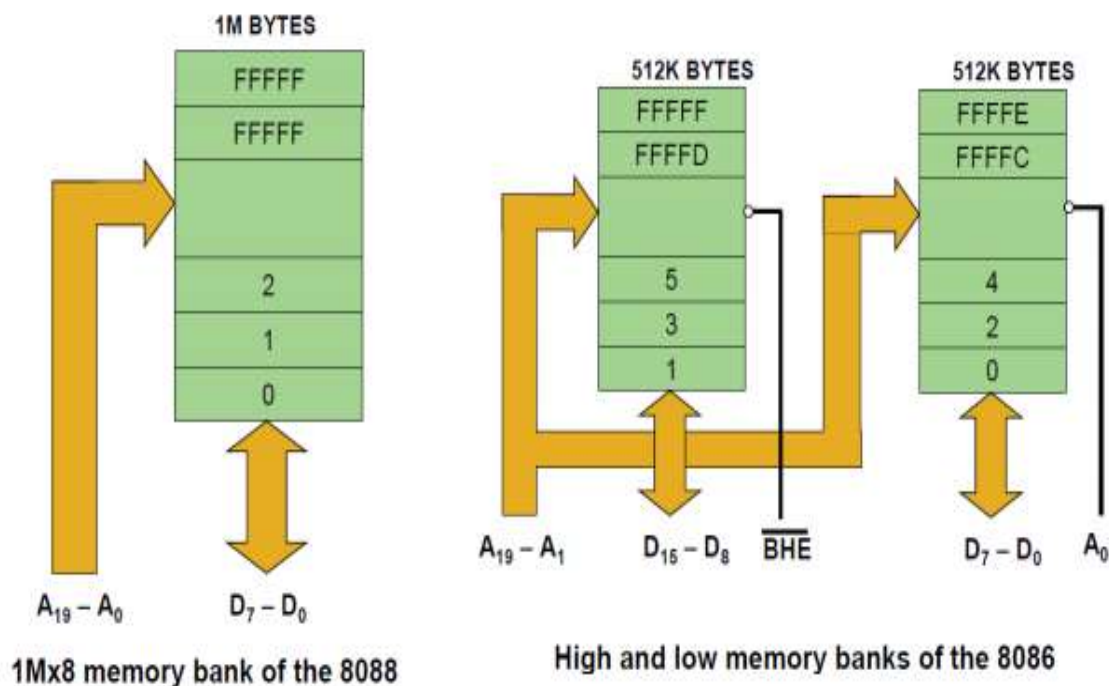
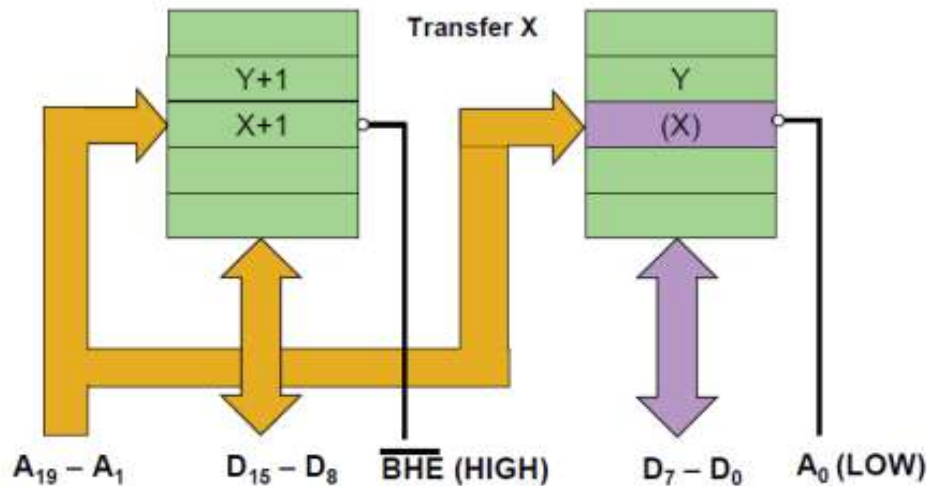
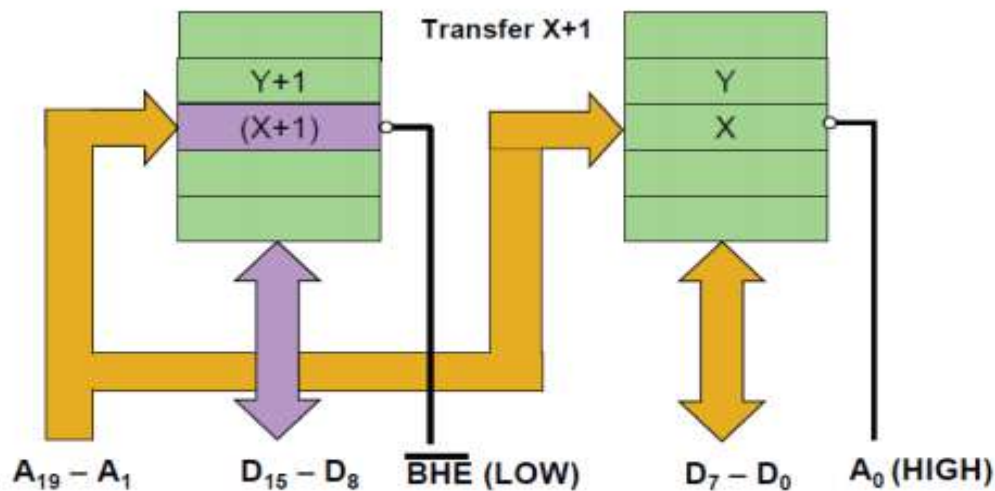


Figure below shows how a byte transfer operation is performed to address X, an even-addressed storage location. A0 is set to logic 0 to enable the low bank of memory and $\overline{\text{BHE}}$ to logic 1 to disable the high bank. Data are transferred to or from the lower bank over data bus lines D0 - D7.



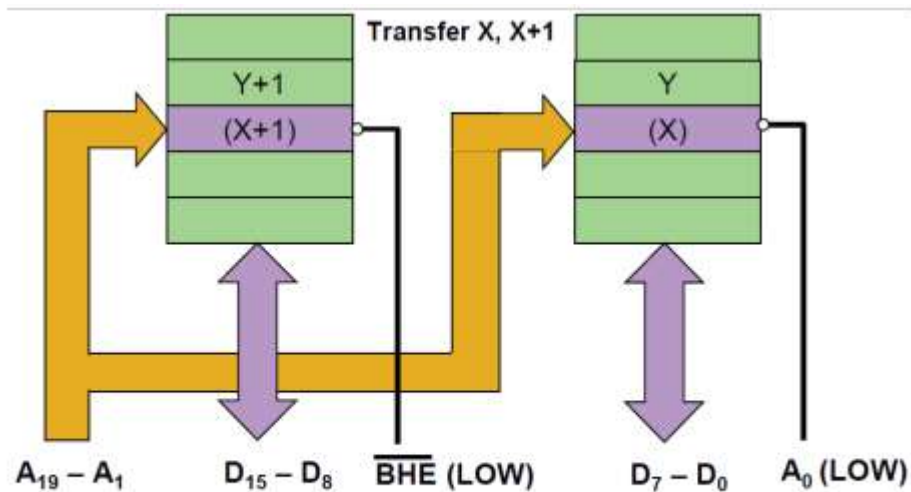
Even address byte transfer by the 8086

Figure below shows how a byte transfer operation is performed to an odd addressed storage location such as $X + 1$. A_0 is set to logic 1 and BHE to logic 0. This enables the high bank of memory and disables the low bank. Data are transferred over bus lines D_8 through D_{15} . D_8 represents the LSB.



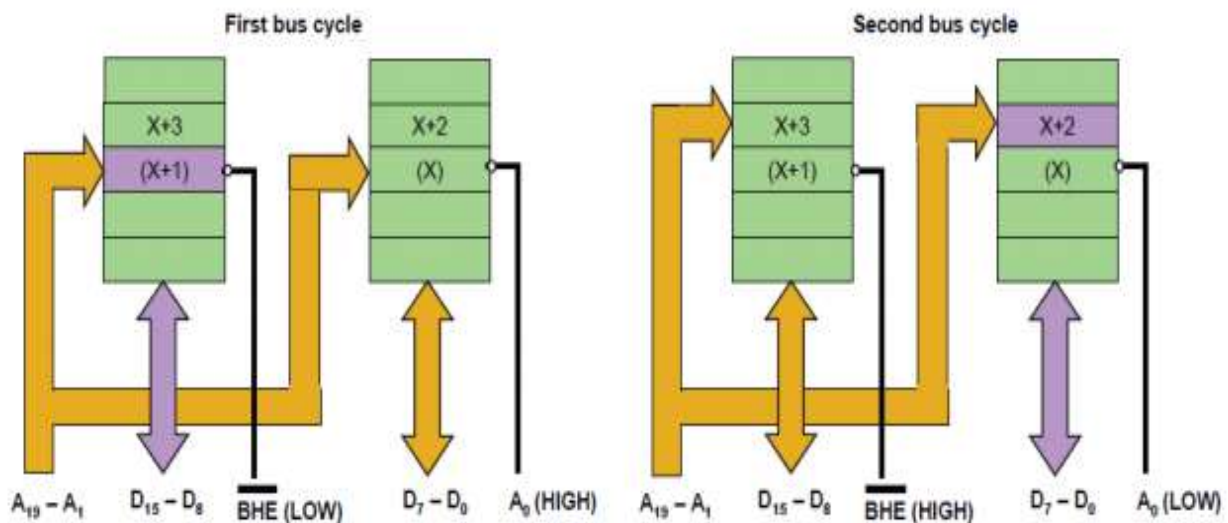
Odd address byte transfer by the 8086

Figure below illustrates how an aligned word (at even address X) is accessed. Both the high and low banks are accessed at the same time. Both A_0 and BHE are set to 0. This 16-bit word is transferred over the complete data bus D_0 through D_{15} in just **one bus cycle**.



Even address word transfer by the 8086

Figure below illustrates how a misaligned word (at address $X + 1$) is accessed. **Two bus cycles are needed**. During the first bus cycle, the byte of the word located at address $X + 1$ in the high bank is accessed over D8 through D15. Even though the data transfer uses data lines D8 through D15, to the processor it is the low byte of the addressed data word. In the second memory bus cycle, the even byte located at $X + 2$ in the low bank is accessed over bus lines D0 through D7.





CYBER SECURITY TECHNOLOGY ENGINEERING DEPARTMENT

MICROPROCESSOR SECOND STAGE

Lecture.7



Asst. Lect. Haider Saad

2024 - 2025

Introduction to 8086 Assembly Language Programming

-What Is 8086 Assembly Language?

8086 assembly language is a low-level programming language that directly communicates with the hardware using mnemonics (instruction codes) like MOV, ADD, and INT. It offers control over hardware resources, making it ideal for learning how CPUs execute instructions.

-What Is An Assembler?

Assembler is a system level program that translates assembly language source code to machine language.

Assembler contains two files:

Object file - contains machine instructions, initial data, and information used when loading the program.

Listing file - contains a record of the translation process, addresses, generated code and data, and a symbol table.

-Why Learn Assembly Language?

- Learn how a processor works.
- Understand basic computer architecture.
- Explore the internal representation of data and instructions.
- Allows creation of small and efficient programs.

Machine Language A language of numbers, called the Processor's Instruction Set, each instruction is coded as a number, instructions may be one or more bytes, every number corresponds to an instruction.

Assembly Language vs Machine Language Programming

- **Machine Language Programming**

Writing a list of numbers representing the bytes of machine instructions to be executed and data constants to be used by the program.

- **Assembly Language Programming**

Using symbolic instructions to represent the data that will form the machine language program and initial data constants.

8086 Instruction - Basic Structure

<i>Operator</i>	<i>Operand[s]</i>	<i>;Comment</i>
-----------------	-------------------	-----------------

Operator - assembly language instruction

Mnemonic: an instruction format for humans

Assembler translates mnemonic into hexadecimal opcode

example: mov is f8h

Operand[s] - 0 to 3 pieces of data required by instruction

Can be several different forms: immediate, register name, memory data, and memory address.

Comment - Extremely useful in assembler language.

8086 Instruction - Example

```
mov    ax, bx    ; Copy contents of bx into ax
```

Operator - mov

Operands - ax and bx

Comment - alphanumeric string between; and \n

INSTRUCTION SET OF 8086

The 8086 instructions are categorized into the following main types:

1. **Data copy /transfer instructions:** These type of instructions are used to transfer data from source operand to destination operand.
2. **Arithmetic and Logical instructions:** All the instructions performing arithmetic, logical, increment, decrement, compare and ASCII instructions belong to this category.
3. **Branch Instructions:** These instructions transfer control of execution to the specified address.
4. **Loop instructions:** These instructions can be used to implement unconditional and conditional loops.
5. **Machine control instructions:** These instructions control the machine status.
6. **Flag manipulation instructions:** All the instructions which directly effect the flag register come under this group of instructions.
7. **Shift and Rotate instructions:** These instructions involve the bit wise shifting or rotation in either direction with or without a count in CX.
8. **String manipulation instructions:** These instructions involve various string manipulation operations like Load, compare, store etc...



CYBER SECURITY TECHNOLOGY ENGINEERING DEPARTMENT

MICROPROCESSOR SECOND STAGE

Lecture.8



Asst. Lect. Haider Saad

2024 - 2025

Data Copy/ Transfer Instructions:

The following instructions come under data copy / transfer instructions:

MOV	PUSH	POP	IN	OUT	PUSHF	POPF	LEA	LDS/LES	XLAT
XCHG	LAHF	SAHF							

Data Copy/ Transfer Instructions:

MOV: MOVE: This data transfer instruction transfers data from one register / memory location to another register / memory location. The source may be any one of the segment register or other general purpose or special purpose registers or a memory location and another register or memory location may act as destination.

Syntax: 1) MOV mem/reg1, mem/reg2

 [mem/reg1] $\leftarrow$ [mem/reg2]

Ex: MOV BX, 0210H
 MOV AL, BL
 MOV [SI], [BX] $\rightarrow$ is not valid

Memory uses DS as segment register. No memory to memory operation is allowed. It won't affect flag bits in the flag register.

2) MOV mem, data
 [mem] $\leftarrow$ data

Ex: MOV [BX], 02H
 MOV [DI], 1231H

3) MOV reg, data
 [reg] $\leftarrow$ data

Ex: MOV AL, 11H
 MOV CX, 1210H

4) MOV A, mem
 [A] $\leftarrow$ [mem]

Ex: MOV AL, [SI]
 MOV AX, [DI]

5) MOV mem, A
 [mem] $\leftarrow$ A
 A $\leftarrow$: AL/AX

Ex: MOV [SI], AL
 MOV [SI], AX

6) MOV segreg, mem/reg
 [segreg] $\leftarrow$ [mem/reg]

Ex: MOV SS, [SI]

7) MOV mem/reg, segreg
 [mem/reg] $\leftarrow$ [segreg]

Ex: MOV DX, SS

In the case of immediate addressing mode, a segment register cannot be destination register. In other words, direct loading of the segment registers with immediate data is not permitted. To load the segment registers with immediate data, one will have to load any general-purpose register with the data and then it will have to be moved to that particular segment register.

Ex: Load DS with 5000H

1) MOV DS, 5000H; Not permitted (invalid)

Thus to transfer an immediate data into the segment register, the convert procedure is given below:

```
2)      MOV AX, 5000H
        MOV DS, AX
```

Both the source and destination operands cannot be memory locations (Except for string instructions)

Other MOV instructions examples are given below with the corresponding addressing modes.

```
3)      MOV AX, 5000H;      Immediate
4)      MOV AX, BX;         Register
5)      MOV AX, [SI];       Indirect
6)      MOV AX, [2000H];     Direct
7)      MOV AX, 50H[BX];     Based relative, 50H displacement
```

PUSH: Push to Stack: This instruction pushes the contents of the specified register/memory location on to the stack. The stack pointer is decremented by 2, after each execution of the instruction. The actual current stack-top is always occupied by the previously pushed data. Hence, the push operation decrements SP by two and this store the two-byte contents of the operand onto the stack. The higher byte is pushed first and then the lower byte. Thus out of the two decremental stack addresses the higher byte occupies the higher address and the lower byte occupies the lower address.

Syntax:

```
PUSH reg
[SP] ← [SP]-2
[[S]] ← [reg]
```

Ex:

- 1) PUSH AX
- 2) PUSH DS
- 3) PUSH [5000H]; content of location 5000H & 5001H in DS are pushed onto the stack.

POP: Pop from stack: This instruction when executed, loads the specified register / memory location with the contents of the memory location of which address is formed using the current stack segment and stack pointer as usual. The stack pointer is incremented by 2. The POP instruction serves exactly opposite to the PUSH instruction.

Syntax:

- i) POP mem
[SP] ← [SP] + 2
[mem] ← [[SP]]
- ii) POP reg
[SP] ← [SP] + 2
[reg] ← [[SP]]

Ex:

1. POP AX
2. POP DS
3. POP [5000H]

XCHG: Exchange: This instruction exchanges the contents of the specified source and destination operands, which may be registers or one of them may be a memory location. However, exchange of data contents of two memory locations is not permitted.

Syntax:

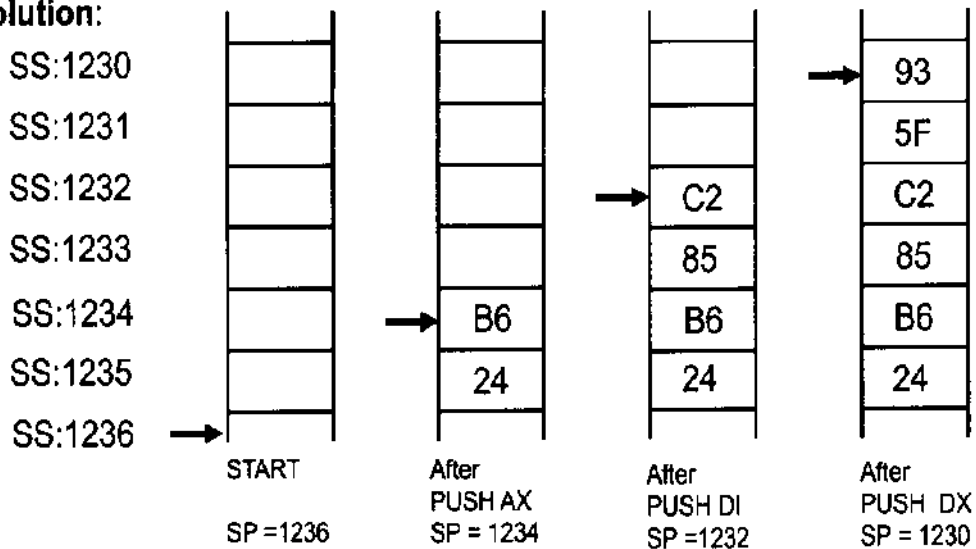
- i) XCHG AX, reg 16
[AX] ↔ [reg 16]
Ex: XCHG AX, DX
- ii) XCHG mem, reg
[mem] ↔ [reg]
Ex: XCHG [BX], DX

Example 1-6

Assuming that $SP = 1236$, $AX = 24B6$, $DI = 85C2$, and $DX = 5F93$, show the contents of the stack as each of the following instructions is executed:

PUSH AX
PUSH DI
PUSH DX

Solution:

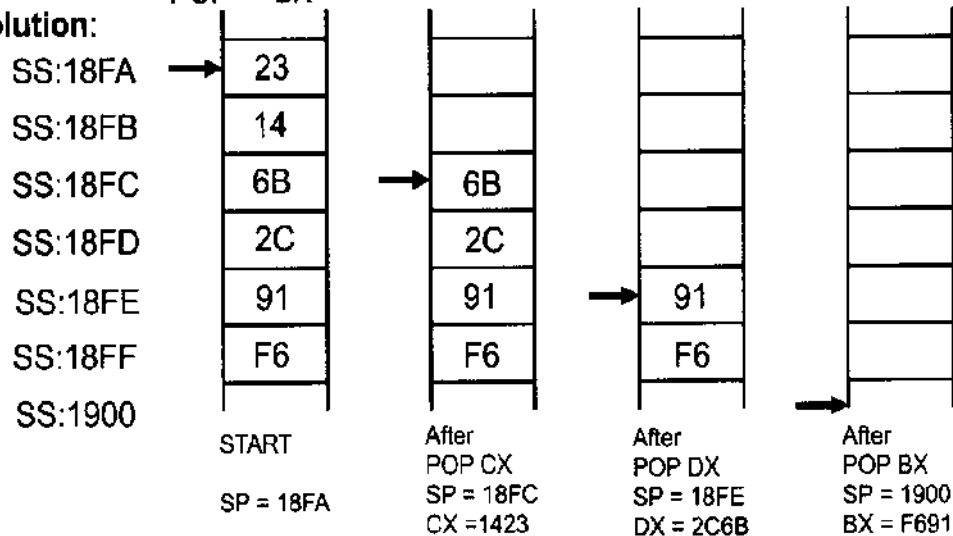


Example 1-7

Assuming that the stack is as shown below, and $SP = 18FA$, show the contents of the stack and registers as each of the following instructions is executed:

POP CX
POP DX
POP BX

Solution:



Register and memory can be both 8-bit and 16-bit and memory uses DS as segment register.

iii) XCHG reg, reg
[reg] ↔ [reg]

Ex: XCHG AL, CL
XCHG DX, BX

Other examples:

XCHG [5000H], AX; This instruction exchanges data between AX and a memory location [5000H] in the data segment.

I/O Operations:

IN: Input the port: This instruction is used for reading an input port. The address of the input port may be specified in the instruction directly or indirectly AL and AX are the allowed destinations for 8 and 16-bit input operations. DX is the only register (implicit), which is allowed to carry the port address.

Ex: 1. IN AL, DX

[AL] ← [PORT DX]

Input AL with the 8-bit contents of the port addressed by DX

2. IN AX, DX

[AX] ← [PORT DX]

3. IN AL, PORT

[AL] ← [PORT]

4. IN AX, PORT

[AX] ← [PORT]

5. IN AL, 0300H; This instruction reads data from an 8-bit port whose address is 0300H and stores it in AL.

6. IN AX ; This instruction reads data from a 16-bit port whose address is in DX (implicit) and stores it in AX.

OUT: Output to the Port: This instruction is used for writing to an output port. The address of the output port may be specified in the instruction directly or implicitly in DX. Contents of AX or AL are transferred to a directly or indirectly addressed port after execution of this instruction. The data to an odd addressed port is transferred on D₈–D₁₅ while that to an even addressed port is transferred on D₀–D₇. The registers AL and AX are the allowed source operands for 8-bit and 16-bit operations respectively.

Ex: 1. OUTDX,AL

[PORT DX] ← [AL]

2. OUT DX,AX

[PORT DX] ← [AX]

3. OUT PORT,AL

[PORT] ← [AL]

4. OUT PORT,AX

[PORT] ← [AX]

Output the 8-bit or 16-bit contents of AL or AX into an I/O port addressed by the contents of DX or local port.

5. OUT 0300H,AL; This sends data available in AL to a port whose address is 0300H

6. OUT AX; This sends data available in AX to a port whose address is specified implicitly in DX.



CYBER SECURITY TECHNOLOGY ENGINEERING DEPARTMENT

MICROPROCESSOR SECOND STAGE

Lecture.9



Asst. Lect. Haider Saad

2024 - 2025

Arithmetic Instructions

ADD	ADC	SUB	SBB	MUL	IMUL	DIV	IDIV	CMP	NEGATE
INC	DEC	DAA	DAS	AAA	AAS	AAM	AAD	CBW	CWD

These instructions usually perform the arithmetic operations, like addition, subtraction, multiplication and division along with the respective ASCII and decimal adjust instructions. The increment and decrement operations also belong

to this type of instructions. The arithmetic instructions affect all the conditional code flags. The operands are either the registers or memory locations immediate data depending upon the addressing mode.

ADD: Addition: This instruction adds an immediate data or contents of a memory location specified in the instruction or a register (source) to the contents of another register (destination) or memory location. The result is in the destination operand. However, both the source and destination operands cannot be memory operands. That means memory to memory addition is not possible. Also the contents of the segment registers cannot be added using this instruction. All the condition code flags are affected depending upon the result.

Syntax: i. ADD mem/reg1, mem/reg2
 [mem/reg1] ← [mem/reg1] + [mem/reg2]

Ex : ADD BL, [ST]
 ADD AX, BX

- ii. ADD mem, data
 [mem] ← [mem] + data

Ex:
ADD [SI], 0712H

- iii. ADD reg, data
 $[reg] \leftarrow [reg] + data$

Ex: ADD CL, 05H
 ADD DX, 0132H

iv. ADD A, data
 $[A] \leftarrow [A] + \text{data}$

Ex: ADD AL, 02H
 ADD AX, 1211H

Examples with addressing modes:

1. ADD AX, 0100H

Immediate

2. ADD AX, BX

Register

3. ADD AX, [SI]

Register Indirect

4. ADD AX, [5000H]

Direct

5. ADD [5000H], 0100H

Immediate

6. ADD 0100H

Destination AX (implicit)

ADC: Add with carry: This instruction performs the same operation as ADD instruction, but adds the carry flag bit (which may be set as a result of the previous calculations) to the result. All the condition code flags are affected by this instruction.

Syntax:

- i. $\text{ADC mem/reg1, mem/reg2}$
 $[\text{mem/reg1}] \leftarrow [\text{mem/reg1}] + [\text{mem/reg2}] + \text{CY}$

Ex: ADC BL, [SI]
 ADC AX, BX

- ii. ADC mem, data
 $[\text{mem}] \leftarrow [\text{mem}] + \text{data} + \text{CY}$

Ex: ADC [SI], 0712H

- iii. ADC reg, data

$[\text{reg}] \leftarrow [\text{reg}] + \text{data} + \text{CY}$
Ex: ADC AL, 02H
 ADC AX, 1211H

Examples with addressing modes:

- | | |
|-----------------------|-------------------------|
| 1. ADC 0100H | Immediate (AX implicit) |
| 2. ADC AX, BX | Register |
| 3. ADC AX, [SI] | Register indirect |
| 4. ADC AX, [5000H] | Direct |
| 5. ADC [5000H], 0100H | Immediate |

SUB: Subtract: The subtract instruction subtracts the source operand from the destination operand and the result is left in the destination operand. Source operand may be a register or a memory location, but source and destination operands both must not be memory operands. Destination operand cannot be an immediate data. All the condition code flags are affected by this instruction.

Syntax:

- i. Sub mem/reg1, mem/reg2
 $[\text{mem/reg1}] \leftarrow [\text{mem/reg1}] - [\text{mem/reg2}]$

Ex: SUB BL, [SI]
 SUB AX, BX

- ii. SUB mem, data
 $[\text{mem}] \leftarrow [\text{mem}] - \text{data}$

Ex: SUB [SI], 0712H

- iii. SUB A, data
 $[\text{A}] \leftarrow [\text{A}] - \text{data}$

Ex: SUB AL, 02H
 SUB AX, 1211H

Examples with addressing modes:

- | | |
|----------------------|----------------------------|
| 1. SUB 0100H | Immediate [destination AX] |
| 2. SUB AX, BX | Register |
| 3. SUB AX, [5000H] | Direct |
| 4. SUB [5000H], 0100 | Immediate |

SBB: Subtract with Borrow: The subtract with borrow instruction subtracts the source operand and the borrow flag (CF) which may reflect the result of the previous calculations, from the destination operand. Subtraction with borrow, here means subtracting 1 from the subtraction obtained by SUB, if carry (borrow) flag is set.

The result is stored in the destination operand. All the conditional code flags are affected by this instruction.

Syntax:

- i. **SBB mem/reg1, mem/reg2**
 $[mem/reg1] \leftarrow [mem/reg1] - [mem/reg2] - CY$
Ex: SBB BL, [SI]
SBB AX, BX
- ii. **SBB mem, data**
 $[mem] \leftarrow [mem] - data - CY$
Ex: SBB [SI], 0712H
- iii. **SBB reg, data**
 $[reg] \leftarrow [reg] - data - CY$
Ex: SBB CL, 05H
SBB DX, 0132H
- iv. **SBB A, data**
 $[A] \leftarrow [A] - data - CY$
Ex: SBB AL, 02H
SBB AX, 1211H

INC: Increment: This instruction increments the contents of the specified register or memory location by 1. All the condition flags are affected except the carry flag CF. This instruction adds 1 to the content of the operand. Immediate data cannot be operand of this instruction.

Syntax:

- i. **INC reg16**
 $[reg\ 16] \leftarrow [reg\ 16] + 1$
Ex: INC BX
- ii. **INC mem/reg 8**
 $[mem] \leftarrow [mem] + 1$
 $[reg\ 8] \leftarrow [reg\ 8] + 1$
Ex: INC BL
INC SI

Segment register cannot be incremented. This operation does not affect the carry flag.

- | | |
|----------------|---------------------------------|
| | Examples with addressing modes: |
| 1. INC AX | Register |
| 2. INC [BX] | Register indirect |
| 3. INC [5000H] | Direct |

DEC: Decrement: The decrement instruction subtracts 1 from the contents of the specified register or memory location. All the condition code flags except carry flag are affected depending upon the result. Immediate data cannot be operand of the instruction.

- Syntax:**
- i. DEC reg16
[reg 16] ← [reg 16] - 1
Ex: DEC BX
 - ii. DEC mem/reg8
[mem] ← [mem] - 1
[reg 8] ← [reg 8] - 1
Ex: DEC BL

Segment register cannot be decremented.

Examples with addressing mode:

- 1. DEC AX Register
- 2. DEC [5000H] Direct

MUL: Unsigned multiplication Byte or Word: This instruction multiplies unsigned byte or word by the content of AL. The unsigned byte or word may be in any one of the general-purpose register or memory locations. The most significant word of result is stored in DX, while the least significant word of the result is stored in AX. All the flags are modified depending upon the result. Immediate operand is not allowed in this instruction. If the most significant byte or word of the result is '0' IF and OF both will be set.

Syntax: MUL mem/reg

- For 8X8
[AX] ← [AL] * [mem8/reg8]
Ex: MUL BL
[AX] ← [AL] * [BL]
- For 16X16
[DX][AX] ← [AX] * [mem16/reg16]
Ex: MUL BX
[DX][AX] ← [AX] * [BX]
- ↓ ↓
higher lower
16-bit 16-bit
- Ex:** 1. MUL BH ; [AX] ← [AL] * [BH]
 2. MUL CX ; [DX][AX] ← [AX] * [CX]

IMUL: Signed Multiplication: This instruction multiplies a signed byte in source operand by a signed byte in AL or signed word in source operand by signed word in AX. The source can be a general purpose register, memory operand, index register or base register, but it cannot be an immediate data. In case of 32-bit results, the higher order word (MSW) is stored in DX and the lower order word is stored in AX. The AF, PF, SF and ZF flags are undefined after IMUL.

Syntax: IMUL mem/reg
For 8X8
 $[AX] \leftarrow [AL] * [mem8/reg8]$
Ex: IMUL BL
 $[AX] \leftarrow [AL] * [BL]$

For 16X16
 $[DX][AX] \leftarrow [AX] * [mem16/reg16]$
Ex: IMUL BX
 $[DX][AX] \leftarrow [AX] * [BX]$

Memory or register can be 8-bit or 16-bit and this instruction will affect carry flag & overflow flag.

Ex: 1. IMUL BH
2. IMUL CX
3. IMUL [SI]

DIV: Unsigned division: This instruction performs unsigned division. It divides an unsigned word or double word by a 16-bit or 8-bit operand. The dividend must be in AX for 16-bit operation and divisor may be specified using any one of the addressing modes except immediate. The result will be in AL (quotient) while AH will contain the remainder. If the result is too big to fit in AL, type 0 (divide by zero) interrupt is generated. In case of a double word dividend (32-bit), the higher word should be in DX and lower word should be in AX. The divisor may be specified as already explained. The quotient and the remainder, in this case, will be in AX and DX respectively. This instruction does not affect any flag.

Syntax: DIV mem/reg

Ex: DIV BL (i.e. $[AX]/[BX]$)

For $16 \div 8$ $\frac{[AX]}{[mem\ 8/reg\ 8]} \rightarrow$ $[AH] \leftarrow \text{Remainder}$
 $[AL] \leftarrow \text{Quotient}$

For $32 \div 16$ $\frac{[DX][AX]}{[mem\ 16/reg\ 16]} \rightarrow$ $[DX] \leftarrow \text{Remainder}$
 $[AX] \leftarrow \text{Quotient}$

Ex: DIV BX $\left(\text{i.e. } \frac{[DX][AX]}{[BX]} \right)$

IDIV: Signed Division: This instruction performs same operation as the DIV instruction, but it with signed operands the results are stored similarly as in case of DIV instruction in both cases of word and double word divisions the results will also be signed numbers. The operands are also specified in the same way as DIV instruction. Divide by zero interrupt is generated, if the result is too big to fit in AX (16-bit dividend operation) or AX and DX (32-bit dividend operation) all the flags are undefined after IDIV instruction.